Subject: SQL
timing -->4PM-6PM
in week--> mon-sat
duration--> 30-35 days

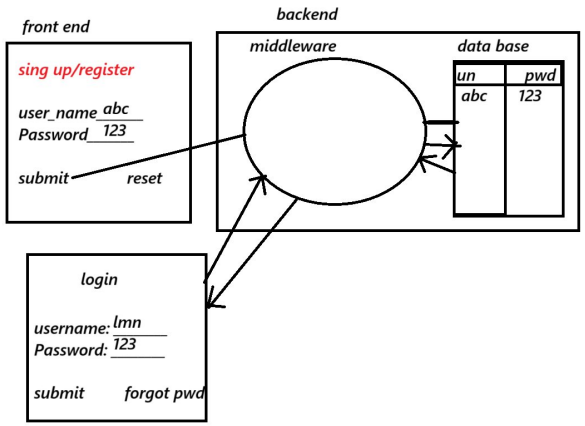
notes will be provided every day.

QTALK--> notes will be share via qtalk

Syllabus will share in qtalk

Assignment:

How SQL is related to Software development
How SQL is related to Software Testing
How SQL is related to Data analyst/Data science.



attend the classes regularly:
---> ask the doubt then and there
--->Assignments has to be done on time. and submit it in Qtalk

if a technical problem-->Technical team
if problem regarding course--> counsellors
if problem is regarding requirements/interview---> HR's
if problem regarding the subject---> Trainers

SQL(Structure Query Language)

History of SQL:

Sql is given by the Donald D Chamberlin and Raymond Boyce.
--> in the early 1970's it is introduced
---> the previous name of SQL is SEQUEL(Simple English Query Language)

why SQL is changed from SQL to SEQUEL.

because of Trademark issue the name changed from SEQUEL to SQL.
SEQUEL system Program language IBM SEQUEL is QUERY Language

To solve the conflict between 2 companies their comes ANSI(American National Standard Institute)
so ANSI Suggested to Drop Vowels from SEQUEL----> SQL

SQL is the query language.

What is the difference between Programming Language and Query Language.*** ASSIGNMENT

Always SQL deals with the data.

What is DATA?

Data is a raw fact which describes attributes of an entity.



student.

properties/attributes	Raw facts
Name:	Ankit
Rollno:	10
Phone:	7204026002
gender:	Male

rawfacts: These are unchanged values

Attributes:

These are the properties that describes an entity.

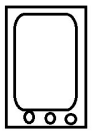
entity:

Anything that exists in the real world are known as Entity.

SQL: (Structured Query Language)
--->Donald D. Chamberlin and Raymond Boyce in the early 1970
-->The previous name of SQL was SEQUEL
--> Because trademark issue they changed the name SEQUEL TO SQL
--> ANSI solve this problem by dropping only vowels the SEQUEL

SQL always deals with the data.

DATA:
It is a rawfact that describes attributes of an entity.



Mobile

Properties/attributes	rawfacts
brand :	SAMSUNG
color :	silver
Price :	1.25L
RAM :	256 GB
Os :	Android
Model:	S24 ultr pro

for the future reference we must store the data.

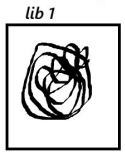
data can be stored in book format, excel, file format, pdf, data base
but in case of SQL we store the form of DATA BASE:

DATA BASE:
It is a place/medium/container in which we store the data in a systematic and organized manner.

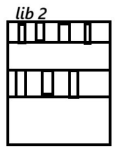


+ Systematic
+organized manner

Why to store the data in a systematic and organized manner.



accessing the book from lib1 is difficult



accessing the book from lib2 is very easy

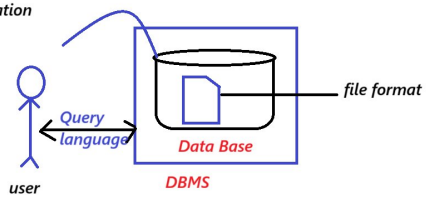
can we store the data randomly on data base.
yes we can store the randomly on data base it is difficult to access the data from data base.

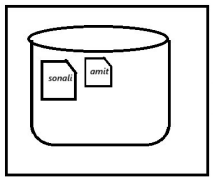
Operations that are performed on data base are known CRUD operations.

C-Create/Insert
R-read/Retrieve
U-Update/Modify
D-Delete/Drop

data base stores the data so, to secure the data we need some software.
and that software is known as DBMS.

DBMS:
Data Base Management System.
It is a software which is used to maintain and manage the data base by providing 2 features
1. Authorization
2. Security.



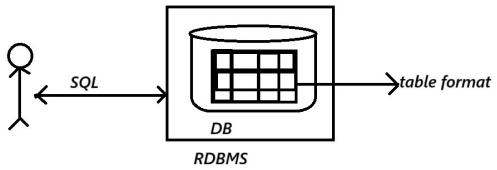


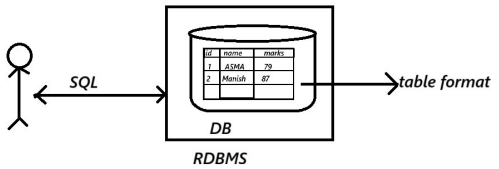
Create operation is difficult to perform on DBMS software
Read operation is also difficult to perform on DBMS software
Update operation is difficult to perform on DBMS software.
Delete operation is also difficult to perform on DBMS software.

--> DBMS stores the data in the form of file format
--> In order to communicate with DBMS we use Query Language.
---> In DBMS it is difficult to perform CRUD operation. and it is the drawback.

So to overcome the drawback of DBMS we use RDBMS

RDBMS: Relational Data Base Management System.
It is a type of software and It is the type of DBMS software which is similar to DBMS.
---> RDBMS is used to maintain and manages the data base. by providing 2 features.
1. Authorization
2. Security





---> RDBMS stores the data in the form of table/tabular format
--->to communicate with table format RDBMS we use SQL language
--> In RDBMS it very easy to perform the CRUD operations.

assignment

Write the difference between DBMS and RDBMS ***

what is R in the RDBMS
R stands of relational

what is Relational.
It is the type of Model is known as Relational Model.

What is relational Model?
It is a type of model which tells storing the data in the form of a table/tabular format/rows and columns
---> This model is given by the great data scientist known as Edger Frank Codd.
-->The idea of storing the data in the form of rows and columns is known as Relation model

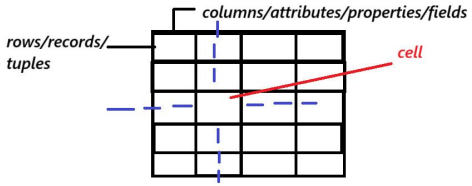
if any DBMS that follows Relational Model -----> RDBMS

or

if any DBMS that follows rules of E.F Codd----> RDBMS

Terminologies related to Relational Model.

Table:
It is the logical organization of rows and columns are known as table.



columns:
It is the single property of an all the entities.

row:
all the properties of single entity is known as row.

Cell:
It is the building block of a table
or
It is the smallest unit of a table
or
It is a space that formed due to intersection of rows and columns.

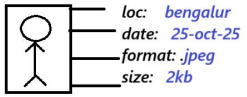
Rules of E.F Codd (total 13 rules)---> 0-12

Rule 1:
According to E.F Codd data stored in a cell must be single valued data.

rollno	name	phone	alt phone
101	Ranjana	6364573210	
102	Ronit	9731562340	8790453210

Rule 2:
In RDBMS we store everything in the form of table including Meta data .

ID	name	photo
1	Jemima	
2	Manish	


loc: bengalur
date: 25-oct-25
format: .jpeg
size: 2kb

Meta data:
Data about the data
or
Further details about the data are known as meta data.

is all the data will have meta data?
no, all the data's will not have meta data only certain data like photos, audios, videos, mp3,mp4,.....

---> Meta data is autogenerated
---> Meta data will be stored in meta table.
---> we can not access/retrieve the meta data from meta table.

rule 3:
According to E.F codd we store data in multiple tables and if need we can establish the connection between 2 tables using key attribute

tid	tname
1	Divya
2	Kavya
3	Navya
4	Shravya
5	Kavya

sid	sname	tid
101	Seema	3
102	Souvik	1
103	Arpita	3

Rule4:
The data entered into the table must be validated in two different steps.

step1:
By assigning the Data Type for the columns (mandatory)

Step2:
By assigning the Constraints.(Not mandatory/option)

empno	ename	sal	job	hiredate	phone
amita	101	null	3000	9870678899	12-feb-2025
20-jun-99	-3000	-3900	Manish	null	978934

Installation of oracle SQL plus software.

link----> bit.ly/roSoftWIN

in setup:
for above windows 10---> windows xp server pack 3

password---> tiger

username: scott
at the end it asks for password manager their unlock Scott and Hr account by the using same password called tiger

DATA TYPES:

It is a type of data that should be entered into the specific column of a table.

---> Data Type is mandatory to assign for all the columns of a table.

Classification of data types.

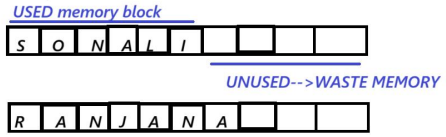
1.CHAR
2.VARCHAR/VARCHAR2
3. NUMBER
4. DATE
5. LARGE OBJECTS
1. CLOB(character Large Objects)
2. BLOB(Binary Large Objects)

1. CHAR:

It is used to store combination of characters like alphabets(A-Z,/a-z),digits(0-9) and special characters(@,&,$....etc)
--> if a character is single/sequence of characters it must be enclosed within single quotes.(' ')
--> CHAR data type can store up to 2000 ch
Syntax:
char(size)

size: It is used to indicate number of characters to be allowed in the specific column.
---> It is used to indicate the length of a data.

Name
Char(10) ---> it accepts 10 characters or less than 10 characters
----



--> In char data type their is a wastage of memory because it follows fixed length memory allocation.

What is fixed length memory allocation:

if any memory which doesn't change or modify that such kind of allocation are called fixed length memory allocation and it is also as static memory allocation.

--> we can use char data type for those columns which accepts same length character for each and every row of entity then for such columns we assign CHAR data Type

example of columns to which we can assign char data type.
PANCARD, VEHICLE,USN, UAN...etc

To overcome the drawback DATA TYPE we use VARCHAR data type.

2. VARCHAR--> Variable character.

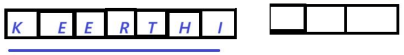
It is the type of data type similar to char data type that is it can also store the combination of characters like alphabets, digits and special characters and that must be enclosed within single quotes.
--> Even VARCHAR data type can store up to 2000 ch.
syntax:
varchar(size)
---> VARCHAR follows Variable length memory allocation

what is Variable length memory allocation:

The length of the memory is going to vary according to the data we store in it. and it is also known as Dynamic Memory allocation

--> In case of Varchar their is no single memory block will be wasted.

name
varchar(10)



example of columns to which we can assign this data type.
password, name,place,address, email, PANCARD

place
varchar(30)

address
varchar(150)

if we want to store more than 2000 character varchar is not sufficient so we use updated version varchar Known Varchar2

VARCHAR2:

It is the type of data type similar to varchar data type it can store combination of characters like alphabets,digits and special symbols withing single quotes.
--> It is also known as updated version varchar data type.
--> Varchar2 can store up to 4000 characters.
---> It also follows dynamic memory allocation
---> their is no wastage of memory

Syntax:

Varchar2(size)

--> By default always compiler consider the varchar2 data type instead of varchar

varchar(15)---> varchar2(15)
varchar(3001)---> varchar2(3001)

article
varchar(3000)
----

example of columns to which we can assign varchar2 data type. name, place, novels, describe

---Write the difference between char,varchar and varchar2 data type.
***

software installation for windows.

Oracle SQL plus.

bit.ly/roSoftWIN

password:---> tiger

at last ---> in password manager unlock scott and hr account by using same password tiger for both scott and hr.

3. NUMBER:

It is the type of data type which is used to store numerical values in a specific column

syntax:

NUMBER(Precision,[SCALE])

PRECISION:

It is used to indicate the integer part of given value.

----> the range of precision is from 1 to 38

---->Precision value is mandatory for number data type

----> Their is no default value for Precision.

SCALE:

It is used store the decimal point value with in the specified precision value.

--->The range of Scale is from -84 to 127

---> Scale value is optional for Number data type

---> The default value for scale is 0

example:

1. Number(5): _ _ _ _ _
   -99999 to +99999
   number(3): _ _ _
   -999 to +999

2. Number(5,2): _ _ _ ._ _
   -999.99 to +999.99
   Number(7,3): _ _ _ _ ._ _ _
   -9999.999 to +9999.999

3. Number(5,5): ._ _ _ _ _
   -0.99999 to +0.99999

4 Number(2,5): _ _ _ _ _
-0 .00999 to +0.00999

example of column to which we can number data type.
phone_no, aadhar, Salary, penalty,.....etc

4. DATE:

It is a type of data type which is used to store date kind of values that be in the oracle given format.

oracle has given 2 format

DD-MON-YYYY

or

DD-MON-YY

---> Always date should be enclosed within single quotes.

example:

'28-OCT-2025' or '28-OCT-25'

Syntax: DATE

example:

DOB, HIREDATE, MFD,EXP_DATE....etc

5. LARGE OBJECTS:

It is the type of data type which is used to store large amount of data.

Types of Large Objects:

1. Character Large Object(CLOB):

It is the type of large object which is used store large amount of characters up to 4GB.

Syntax:

CLOB

2. Binary Large Object(BLOB):

It is used to store large of amount of binary values up to 4 GB data.

Syntax:

BLOB

empid	ename	sal	job	hiredate	comm	phone
number(6)	varchar(20)	number(9,2)	varchar(15)	Date	number(7,2)	number(10)
12345 12345	RESHMA	-1000 8990	CLERK	12-JAN-2025	100	989898394 -12356487

Constraints:

These are the rules are restrictions given for the specific column for extra validation of data .

--->Constraints are optional to assign for the columns.

Types of constraints:

1. NOT NULL
2. UNIQUE
3. CHECK
4. PRIMARY KEY
5. FOREIGN KEY
6. DEFAULT

1. NOT NULL:

null means empty

Not null means not empty.

It is a type of constraint assigned for the columns to make column as mandatory to have the data.

---> If any columns that are assigned with Not null constraints those columns will never accept null/empty values.

syntax: NOT NULL

example:

name, salary,hiredate, empid, job, phone.....

2. UNIQUE:

unique means different /not similar

It is a type of constraint which is used to avoid duplicate or repeated value to the specific column which is assigned Unique.

syntax:

UNIQUE

example:

empid, rollno, pan_no,aadhar, Phone, email....etc

if any column assigned with the unique constraint and if we try to store duplicate data it throws an error saying that unique constraint violated.

3. Check :

It is a type of constraint which is used for extra validating the data. and it must be used along with the condition. if the condition is true it will accepts the data or else it rejects the data.
--> for the single column we can assign more than one check constraint with the different condition.

syntax:

check(condition)

sal
check(sal>0)
---
800 -1000>0 f---> it rejects
800>0 t----> it selects

phone
check(phone>0)
check(length(phone)=10)

9876543210 9876543210>0 t and length(9876543210)=10 t
-9876543768>0 f and length(-9876543768) t
77609834>0 t length(77609834)=10 f

4. Primary Key :

It is a type of constraint which is used to identify the records uniquely from the table.

Syntax: Primary key.

characteristics of Primary Key:
-->Primary key constraint accepts only unique value it is similar to unique constraint.
--> Primary key constraint rejects the null value and it is similar to Not null
-->Primary key constraint is the combination of unique and Not null
-->in a table we have either one primary key
-->their should be only one primary key in a table.

example:
employee: empid
student : rollno
bank :accono

5. Foreign Key:

It is a type of constraint which is used to establish the connection between 2 tables.
---> If any column wants to be foreign key column in another table it has to be Primary key column in its own table.

syntax:

Foreign key

Characteristics of Foreign key:
---> Foreign key accepts duplicate value it is not similar to unique constraint
---> Foreign key accepts null values and it is not similar to Not null constraint
---> FK is not a combination of unique and Not null
---> In a table we can have more than one Foreign key
--->Foreign key is also called as Referential Integrity constraint.

Write the difference between Primary key and Foreign Key constraint ***

teachers

p.k	tid	tname
	1	Aryan
	2	Vinay
	3	Rajath

student

p.k	sid	sname	tid	F.K	cid	f.k
	101	Keerthi	2		c104	
	102	Seema			c101	
	103	Padma	2		c103	
	104	Kush	1		c104	
	105	Vidya	3		c102	

course

cid	cname	cfees
c101	JFD	30000
c102	PFD	40000
c103	Devops	32000
c104	testing	25000

6. Default:

It is a type of constraint which is used to provide the default constraint even we don't enter any value.

syntax:

default default value.

example:

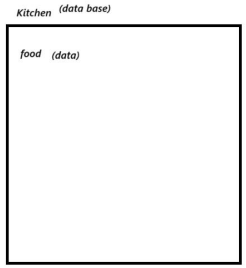
country_code number(4) unique not null default 91

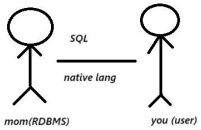
emp

empno	ename	sal	comm	job	hiredate	deptno
number(6)	varchar(25)	number(7,2)	number(5,2)	varchar(25)	date	number(2)
primary key	not null	not null check(sal>0)	check(comm>0)	not null	not null	Foreign key

dept

deptno	dname	loc
number(2)	varchar(25)	varchar(30)
primary key	not null	not null





Overview of SQL:

Types of SQL statements.
1. DDL statements
2. DML statements
3. TCL statements
4. DCL statements
5. DQL statements

1. DDL (DATA DEFINITION LANGUAGE):
It is a type of SQL statement which is used to create and deals the with structure.

2. DML(Data Manipulation Language):
It is a type of SQL statement which is used insert and manipulate the records/data available on the table.

3. TCL(Transaction Control Language):
To control the transactions done by the DML statements we use TCL statements.

4. DCL(Data Control Language):
It is type of SQL statements which are used to control the data flow between 2 users.

5. DQL(Data Query Language):
It is a type of SQL statement which is used to retrieve/fetching/accessing/getting the data from the table.

DQL(Data Query Language):
It is a type of SQL statement which is used to retrieve /fetch the exiting records from the table.

Types of DQL statements:
1. SELECT
2. PROJECTION
3. SELECTION
4. JOINS

1. select:
It is a type of DQL statement which is used to select data/records from the table and display it on the screen.

2. Projection:
It is a type of DQL statement which is used to retrieve the records/data from the table by selecting only columns.

3. Selection:
It is a type of DQL statement which is used to retrieve the records/data from the table by selecting both rows and column.

4. Joins:
It is a type of DQL statement which is used to retrieve the multiple records from multiple table simultaneously.

Projection:
It is a type of DQL statement which is used to retrieve the records/ data of a table by selecting only column.

SYNTAX:
```
select */distinct column_name/Expression [ALIAS]
From Table_Name;
```

clause: It is like a keyword which has some specific task to perform.

select:
It is a type of DQL statement/It is type of clause which is used to select the records from the table and display the records on the screen/console.

from:
It is a type of clause which is used to search for the table from data base and keeps the table under execution.

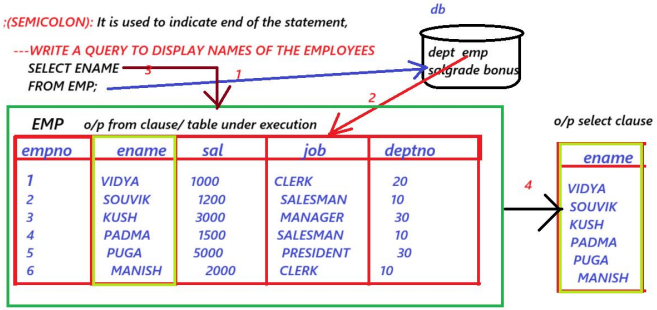
Table_Name:
It is a type of argument passed in the From clause from where we retrieve the records.

Column_Name: It is a type of argument/name of a column passed in the select clause from where select clause selects the records.

order of writing the query is
1. select
2. from

order of execution
1. From
2. select

:(SEMICOLON): It is used to indicate end of the statement,



COMMAND TO CHECK THE TABLES AVAIALABLE ON DATA BASE.
```
SELECT *
FROM TAB;

 TNAME
---------
EMP
DEPT
SALGRADE
BONUS
```

COMMAND TO CHECK COLUMN_NAMES AVAILABLE ON TABLE.
```
DESC TABLE_NAME;

DESC EMP;

DESC DEPT;
```

Projection:

It is a type of DQL statement which is used to retrieve the records from the table by selection only the columns.

Syntax:

```
select */distinct column_name/expression [ALIAS]
from Table_Name;
```

order of execution is different from its order of writing.

order of execution :
1. FROM
2. SELECT

execution of Projection:
--> In all the query always From clause will be executed first
--> The job of from clause is used go to the data base search for the table, if table is available it keeps the table under execution and if table is not available it throws an error saying table or view doesn't exist.
--> After the execution of from clause select clause starts its execution
--> The job of select clause is used to go to the o/p from clause or that table which is under execution and search for the column and selects the column then displays that column as an output. in case column name is not available it throws and error saying invalid identifier.

```
---WAQTD JOB OF THE EMPLOYEES
  select job
   from emp;

---WAQTD NAME AND DESIGNATION OF EMPLOYEES
SELECT ENAME,JOB
FROM EMP;

---WAQTD NAME,SALARY AND  HIREDATE OF AN EMPLOYEES
SELECT ENAME,SAL,HIREDATE
FROM EMP;
```

---WAQTD DETAILS OF EMPLOYEES

*to display all the columns of the table instead of mentioning each and every columns of a table we use another type of argument in a select clause known as *(asterisks)*

**(Asterisks):*

It is a type of argument should be using in the select to display all the columns of a table by default.
*--->Always * should be used independently that means along *(asterisks) we are not suppose to pass any other columns.*

```
select *
from emp;

 --waqtd department name of department table
  select dname
from dept;

---waqtd details of dept table
select *
from dept;
```

DISTINCT clause:

distinct means unique/different

It is a type clause used to remove the duplicate/repeated records from specific column of a table.
--->always distinct should be the first argument in the select clause.
---> for the distinct clause we can pass multiple columns also.

syntax: distinct Column_Name

```
--waqtd diffent salary given to the employees.
  select distinct sal
   from emp;

 ---waqtd different job and salary available on emp table
 select distinct job,sal
 from emp;
```

---WAQTD different depts available on emp table.

```
select distinct deptno
from emp;
```

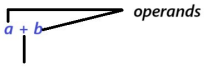
Can we use unique constraint like distinct clause.

Yes we can use unique constraint like distinct clause in a given query.

```
select unique deptno
from emp;
```

EXPRESSION:

It is a combination of operator and operands which gives us a result are known as Expressions.



Operand:

These are the values which acts on operator to perform specific task are known as Operands.

Operator:

It is a symbol which acts on operand to perform a specific task are known as operator.

---WAQTD annual salary(yearly salary) from emp table.

```
select sal*12
from emp;
```

---waqtd name,job and salary with the hike of 2000 rs. from emp table

```
select ename,job,sal+2000
from emp;
```

SQL> ---waqtd name and salary of the employees along with that display the salary with penalty of 200 rs.

```
SQL> select ename,sal,sal-200
  2  from emp;
```

25% of 1000 25/100*1000=250
0.25*1000 =250
1000/4 =250

---waqtd 25% of salary of all the employees

```
select sal*0.25
from emp;
```

```
sal*0.75
sal+sal*0.75
```

SQL> --waqtd name,salary and deptno along with that display the salary with deduction of 50% of their salary.

```
SQL> select ename,sal,deptno,sal-sal*0.5
  2  from emp;
```

---waqtd total salary earned each employees on emp table

```
select sal+comm
from emp;
```

the above query gives incorrect o/p because of null values in the comm column,

here whenever we perform any arithmetic operation using null value it again returns o/p as null. so it will be the drawback of null value.

so we overcome the drawback null value by using NVL().

1000+null=null
200*null=null

---waqtd details of a employees along with that display the annual salary of employees.

```
select *,sal*12
from emp;
```

---> this query gives an error because along * we are not suppose to any other column.

So to overcome the drawback * we use another type of argument called as Table_Name.*.

Table_Name.*:

It is a type of argument passed in the select to get the details of the tables. but the difference is it will accept any column_name or expression along with it.

SQL> --waqtd name,job of the employees along with that display the annual salary with the hike 25% of annual salary

```
select ename,job,(sal*12)+(sal*12)*0.25
from emp;
```

Alias:

It is a alternate name given for column_name,expression,table_name.

--> to enhance the readability of our output we can use this alias name

--> Alias is not mandatory it is optional.

-->Alias name can be used with or without as keyword.

--> if alias name has 2 words and if we want to provide a space between those then it must enclosed withing double quotes

-->if not space we can use _(underscore) between two words of a alias name.

---waqtd name and annual salary of an employees

```
select ename,sal*12 as AnnualSalary
from emp;

 select ename,sal*12 as "Annual salary"
from emp;

select ename,sal*12 as Annual_Salary
from emp;
```

50% of salary ---> sal*50/100

salary with the hike of 50% of a salary: sal+(sal*50/100)

---waqtd details of a employees along with that display the salary with the hike 50% from emp table

```
select emp.*,sal+sal*0.5 "hike of 50%"
from emp;
```

SQL> ---waqtd name,job and salary of the employees along with that display the half term salary

```
SQL> select ename,job,sal,sal*6 Half_term_salary
  2  from emp;
```

SELECTION:

It is a type of DQL statement which is used to retrieve data from the table by selecting both rows and columns.

--- WAQTD job of smith from emp table.

this query can not be solved using Projection syntax so we use Selection syntax to write the query for this question.

Syntax of Selection:

```
SELECT */DISTINCT COLUMN_NAME /EXPRESSION [ALIAS]
FROM TABLE_NAME
WHERE <FILTER_CONDITION>;
```

ORDER OF WRITING

1. SELECT
2. FROM
3. WHERE

ORDER OF EXECUTION THE QUERY.

1. FROM
2. WHERE
3. SELECT

WHERE :

It is a type of a clause which is used to filter out the records by using filter condition.

where clause selects the records of a table if the condition is true if the condition is false the record will be rejected.

filter condition:

It is a condition passed in the where clause to filter the records from the table.

Syntax:

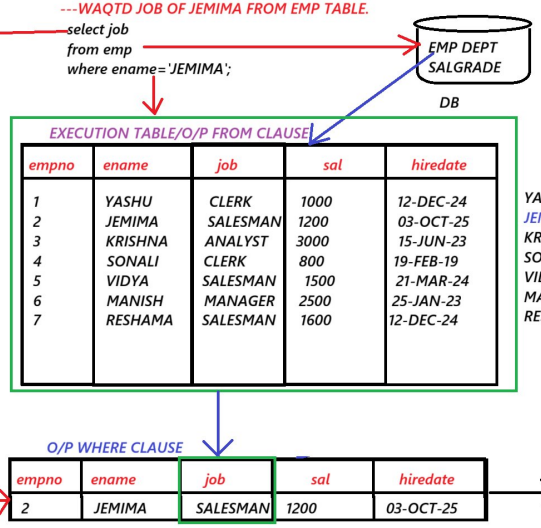
column_name/Expression operator value.

Note:

SQL is case insensitive but the data available on the table are case sensitive.

---WAQTD JOB OF JEMIMA FROM EMP TABLE.

```
select job
from emp
where ename='JEMIMA';
```



empno	ename	job	sal	hiredate
1	YASHU	CLERK	1000	12-DEC-24
2	JEMIMA	SALESMAN	1200	03-OCT-25
3	KRISHNA	ANALYST	3000	15-JUN-23
4	SONALI	CLERK	800	19-FEB-19
5	VIDYA	SALESMAN	1500	21-MAR-24
6	MANISH	MANAGER	2500	25-JAN-23
7	RESHAMA	SALESMAN	1600	12-DEC-24

YASHU=JEMIMA F REJECTED 1000>1000 f
JEMIMA=JEMIMA T SELECTED 1200>1000 t
KRISHNA=JEMIMA F REJECTED 3000>1000 t
SONALI=JEMIMA F REJECTED 800>1000 f
VIDYA=JEMIMA F REJECTED 1500>1000 t
MANISH=JEMIMA F REJECTED 2500>1000 t
RESHMA=JEMIMA F REJECTED 1600>1000 t

empno	ename	job	sal	hiredate
2	JEMIMA	SALESMAN	1200	03-OCT-25

job
SALESMAN

---waqtd names and job of the employees whose salary is greater than 1000

```
select ename,job
from emp
where sal>1000;
```

Note points on where:

--> Where is clause which is used to filter the records
--> In where clause we pass filter condition
--> where clause will be executed after the from clause
-->where clause always executes row by row/record by record to filter the records.
--> we can pass multiple filter condition in where clause by using logical operator.

---WAQTD job of smith from emp

```
select job
from emp
where ename='SMITH';
```

---waqtd details of the employee who are working salesman

```
select *
from emp
where job='SALESMAN';
```

---WAQTD name,job and annual salary of the employees whose annual salary is more than 10000

```
select ename,job,sal*12 annual_sal
from emp
where sal*12>10000;
```

```
---waqtd name and hiredate of the employees who are hired after 1981.
  select ename,hiredate
  from emp
  where hiredate>'31-dec-1981';

or

select ename,hiredate
from emp
where hiredate>='01-jan-1982';

----waqtd details of the employees who are hired before 1982
select *
from emp
where hiredate<'01-jan-1982';

or

select *
from emp
where hiredate<='31-dec-1981';

 ---waqtd name job and salary of the employee except those who are working as clerk.
 select ename,job,sal
 from emp
 where job!='CLERK';

 or

 select ename,job,sal
 from emp
 where job<>'CLERK';

 ---waqtd details of the employees along with that display half term salary except those
who are working in deptno 20
 select emp.*,sal*6
 from emp
 where deptno!=20;

 ---waqtd name and salary of the employees whoa re getting the salary less than or
equals 2900
 select ename,sal
 from emp
 where sal<=2900;
```

Operator:

These are the symbol which acts upon operands to perform a specific tasks are known as operator.

Types of Operators:

1. Arithmetic Operators
2. Relational/comparison operators
3. Concatenation operator
4. Logical Operators
5. Special operator
    1. IN
    2. Not In
    3. Between
    4. Not Between
    5. Is
    6. Is Not
    7. Like
    8. Not Like
6. subquery operators(ALL,ANY,EXISTS,NOT EXISTS)

1. Arithmetic Operators:

These are operator which are used to perform arithmetic operations.

Following are the arithmetic operator.

+(addition
-(subtraction)
* (Multiplication)
/(division)

2. Relational/comparison operators:

These are operators which are used to compare/relation the values/data of a table.

following are the types of relational operators:

< (less than)
>(greater than)
<=(less than or equals)
>=(greater than or equals)
!= or <>(not equals)
= (equals)

3. Concatenation operator:

It is a type of operator which is used to join/combine 2 columns or two string values. or combining Column_Name and string value.
||---> Concatenation operator.

Syntax: string1||string2

```
---WAQTD names of the employee in such a way in which every employees name is prefixed with 'Mr.'
select 'Mr.'||ename
from emp;

---WAQTD the output in such a way that where it going to greet every employees. (Hi good evening smith)
select 'Hi good evening'||' '||ename
from emp;

SQL> ---waqtd the names and salary of the employees in a such way that both ename and salary column will be the part of single column and give some space between name and salary of an employees.
 select ename||' '||sal
 from emp;
```

Logical operator:

These are the types of operators which works on two kind of values either true or false.
---> logical operators are used between the conditions

types of logical operators are as follows.
1. AND
2. OR
3. NOT

1. AND

It is a type of logical operator which returns true when all the conditions are true and it returns false when if any one condition/all the conditions are false.

syntax:

condition1 AND condition2

```
---waqtd name,salary and job of the employees who are working as a manager and there salay should be greater than 1600.
select ename,sal,job
from emp
where job='MANAGER' and sal>1600;

---waqtd details of the employees who are hired after 1980 and must be working in deptno 20
select *
from emp
where hiredate>'31-dec-1980' and deptno=20;

 ---waqtd details of smith and Jones
 select *
 from emp
 where ename='SMITH' AND ENAME='JONES';
This query give no rows selected as an output because at the same time name can not be smith and jones.
```

Note:

whenever multiple values should be compared from the same column using = operator we should never use and operator between those condition.

```
--waqtd name and salary of the employees who are earning the salary more than 1000 and less than 2000
select ename,sal
from emp
where sal>1000 and sal<2000;
```

OR operator:

It is a type of logical operator which returns true when any one condition is true and It returns false when all the conditions are false.

Syntax:

condition1 OR condition2

```
---WAQTD details of the employees in which either employees are working as clerk or they must get the salary less than 1500
select *
from emp
where job='CLERK' or sal<1500;

SQL> ---WAQTD name ,job and salary of the employees whose name is CLARK  or employees must be either working a manager
SQL> select ename,job,sal
  2  from emp
  3  where ename='CLARK' OR JOB='MANAGER';

---WAQTD DETAILS OF THE EMPLOYEES WHO ARE GETTING 1250 3000 AND 5000 AS A SALARY
select *
from emp
where sal=1250 or sal=3000 or sal=5000;
 select ename,job,sal,deptno
 from emp
 where job='CLERK' AND( DEPTNO=10 OR DEPTNO=30);
```

NOTE:

WHENEVER WE USE OR AS WELL AS AND IN THE SAME QUERY ALWAYS ENCLOSE OR CONDITIONS WITHIN THE PARENTHESIS.

```
 ---WAQTD NAMES ,SALARY DEPTNO AND JOB OF THE EMPLOYEE WHO ARE GETTING THE SALARY LESS THAN 1500 AND MUST EITHER WORKING AS SALESMAN OR CLERK
 Select ename,sal,deptno,job
 from emp
 where sal<1500 and (job='SALESMAN' OR JOB='CLERK');
```

NOT:

It is a type logical unary operator which can works on single condition.

---> It is used negate the given input value.

if the condition is false it gives true and if the condition is true and returns as a false.

Syntax:

Not <condition>

example:

```
---WAQTD names of the employees except those who are working in deptno 20
  select ename
  from emp
  where not deptno=20;

---waqtd details of the employees who salary is not greater than 1000
select *
from emp
where not sal>1000;
```

Special Operators:

1. IN:

It is a type of a special operator which is similar to = operator that means it also used compare the values.

-->In operator accepts multiple values at RHS

--->by using this In operator we reduce the number of conditions in the where clause.

--> This operator is used to compare the multiple values which are from the same column

--> This operator works similar to the combination = and OR operator

--> = operator accepts only one value at RHS but IN operator can accept single value /multiple values at RHS.

Syntax:

Column_Name IN(val1,val2,.....valN);

---> parenthesis are mandatory whenever we compare the multiple values at RHS of IN and it is not mandatory whenever we compare we compare a single value.

```
---WAQTD details of ALLEN,CLARK,JAMES from emp
 select *
 from emp
  where ename In ('ALLEN','CLARK','JAMES');
```

SQL> ---waqtd NAMES ,SALARY AND HIRED OF THE EMPLOYEES WHO ARE HIRED AFTER 1980 AND MUST BE WORKING DEPTNO 10 AND 30

```
SQL> SELECT ENAME,SAL,HIREDATE
  2  FROM EMP
  3  WHERE HIREDATE>'31-DEC-1980' AND DEPTNO IN(10,30);
```

```
 ---WAQTD NAME AND SALARY OF THE EMPLOYEES WHO ARE EARNING 1100,1250,3000 AND 5000 ON EMP TABLE
 SELECT ENAME,SAL
 FROM EMP
 WHERE SAL IN (1100,1250,3000,5000);
```

2. NOT IN:

It is a type of special operator similar to IN operator but instead of selecting the values it is used to reject the values mention at the RHS.

Syntax:

Column_Name Not IN (val1,val2,val3....valN);

```
---waqtd name and job of the employees except those who are working a CLERK and salesman
select ename,job
from emp
where job NOT in ('CLERK','SALESMAN');

 ---WAQTD NAME AND SALARY OF THE EMPLOYEES WHO ARE EARNING 1100,1250,3000 AND 5000 ON EMP TABLE
 SELECT ENAME,SAL
 FROM EMP
 WHERE SAL IN (1100,1250,3000,5000);
```

SQL> ---waqtd name and salary,hiredate,job deptno of the employees who are working as clerk, manager and except those who will be working in 30 and 10 dept

```
 select ename,sal,hiredate,job,deptno
 from emp
 where job in('CLERK','MANAGER') AND DEPTNO NOT IN (10,30);

---waqtd name,and salary of the employees whose salary is greater 1000 and less than 3000
select ename,sal
from emp
where sal>1000 and sal<3000;

---waqtd details of the employees who are hired after 1980 and before 1982
select *
from emp
where hiredate>'31-dec-1980' and hiredate<'01-jan-1982';
```

BETWEEN:

It is a type special operator which is used whenever there are ranges of values.

---> between operator works including the range of values

---> In this between operator we can not change the range of values.

some time we have make some calculation on lower range and higher range in order to exclude the range of value according to the condition.

lower range should be incremented by 1
and higher range should be decremented by 1

syntax:

Column_Name Between lower_range and higher_range

when to exclude the range of values.

--> whenever the question has greater than, less than ,before, after such kind of words.

--> if the question of has between keyword

```
---waqtd name and salary of the employees whose salary is greater than 1000 and less than 3000
select ename,sal
from emp
where sal between 1001 and 2999;

 ---waqtd details of the employees who are hired after 1980 and before 1982
 select *
 from emp
 where hiredate between '01-jan-1981' and '31-dec-1981';
```

```
select emp.*,sal-sal*0.1
from emp;
```

---WAQTD details of the employees hired in the year 1982.

```
select *
from emp
where hiredate between '01-jan-1982' and '31-dec-1982';
```

4. Not Between:

It is a type of special operator similar to Between operator but instead of selecting the range of values it is used to reject the range of values.

Syntax:

Column_Name Not Between lower_range and higher_range;

---WAQTD details of the employees except those who are earning more than 1000 and less than 2500

```
select *
from emp
where sal not between 1001 and 2499;
```

SQL> ----waqtd name salary and job of the employees except those who are earning the salary ranging from 2000 to 3000

```
SQL> select ename,sal,job
  2  from emp
  3  where sal not between 2000 and 3000;
```

QL> ----waqtd name,salary hiredate and deptno of the employees except those who are hired in the year 1981

```
QL> select ename,sal,hiredate,deptno
 2  from emp
 3  where hiredate not  between '01-jan-1981' and '31-dec-1981';
```

---WAQTD names and salary of the employees who are not getting any commission

```
select ename,sal
from emp
where comm=null;
```

this query gives no rows selected as an output because it = operator never compares with null values.

SO to compare with the null values we use special operator known as IS operator

IS operator:

It is a type of a special operator which is used to select the null values.

---> Is operator must be used along with the Null values only

Syntax:

Column_Name Is Null.

---WAQTD names and salary of the employees who are not getting any commission

```
 select ename,sal
  from emp
  where comm is null;
```

SQL> ---waqtd name,sal,job of the employees who don't know whats their designation is

```
SQL> select ename,sal,job
  2  from emp
  3  where job is null;
```

no rows selected

---waqtd details of the employee who doesn't have manager to report

```
 select *
  from emp
  where mgr is null;
```

IS NOT:

It is a type of special operator similar to Is operator but instead of selecting the null values it is used reject the null values.

---> This operator also must be used along with the null value.

Syntax:

Column_Name Is Not Null.

---waqtd details of the employees who are getting the commission

```
select *
from emp
where comm is not null;
```

---waqtd name ad salary of the employees who are getting the salary

```
 select ename,sal
 from emp
 where sal is not null;
```

---WAQTD names of the employees whose name starts with S letter .

```
select ename
from emp
where ename='S';
```

---> no rows selected as an output because = operator search for name called S.

so for the pattern matching we use another type of special operator known as LIKE operator

LIKE operator:

It is a type of a special operator which is used for pattern matching.

based on the pattern it selects the data from the table.

to write the pattern we make use of wild card operator,

below operators are Wild card Operators

1. %(percentie)
2 _(underscore)

Syntax:

Column_Name Like 'Pattern_to_match';

---> here, always pattern to match should be enclosed withing single quotes.

Percentile(%):

It is used to replace any number of character

It is used to replace any character

It is used to replace even no character as well

Underscore(_)

---> It is always used to replace a single character at a time

--> It replaces any character.

---waqtd names of the employees who has a character a in their name (consider all the letters are in upper case)

ename				
Padma	P adma -->r	Padm a-->sel	PADMA	P A DMA --->SEL
Arpita	A rpita---->r	Arpit a-->sel	ARPITA	A RPITA--->SEL
Vidya	V idya------>r	Vidy a---->sel	VIDYA	VIDY A -->SEL
Reshma	R eshma--->r	Reshm a--->sel	RESHMA	RESHM A -->SEL
Sayyed	S ayyed--->sel	Sayyed --->rej	SAYYED	S A YYED--->SEL
Souvik	S ouvik---> sel	Souvik ---->rej	SOUVIK	SOUVIK--->REJ
Krishna	K rishna--->r	Krishn a--->sel	KRISHNA	KRISHN A -->SEL
Sonali	S onali---->sel	sonali --->rej	SONALI	SON A LI--->SEL
Aparna	A parna--->r	seem a--->sel	APARNA	A PARNA-->SEL
Seema	S eema--->sel	% a	SEEMA	SEEM A -->SEL
	S %			% A %

---WAQTD NAMES OF THE EMPLOYEES WHO ARE 2 CHARACTERS OF 'A' IN THEIR NAME

```
select ename
from emp
where ename like '%A%A%';
```

---WAQTD NAMES OF THE EMPLOYEES WHO HAS CONSECUTIVE Y IN THEIR NAME.

```
 SELECT ENAME
FROM EMP
WHERE ENAME LIKE '%YY%';
```

---WAQTD names of the employees whose name starts with J letter

```
select ename
from emp
where ename like 'J%';
```

---waqtd details of the employees whose name ends with 'R'

```
select *
from emp
where ename like '%R';
```

----waqtd name job of the employees whose name 2 letters of R in their name

```
select ename,job
from emp
where ename like '%R%R%';
```

_(underscore)

---waqtd names of the employees whose name has 2 letter as 'A' in their name

```
select ename
from emp
where ename like '_A%';
```

---waqtd details of the employees whose name 3rd letter as I in their name

```
select *
from emp
where ename like '__I%';
```

SQL> ---waqtd name ,salary and job of the employees who name has last but least as a E in their name

```
SQL> select ename ,sal,job
  2  from emp
  3  where ename like '%E_';
```

---waqtd names and hiredate of the employees who are hired in the month DEC

```
select ename,hiredate
from emp
where hiredate like '%DEC%';
```

---waqtd name and salary of the employees who are earning 3 digit salary

```
select ename,sal
from emp
where sal like '___';
```

NOT LIKE operator:

It is a type of special operator and similar to the Like operator but instead of selecting the pattern it is used to reject the pattern.

Syntax:

Column_Name Not like 'Pattern_to_match'

---waqtd details of the employees except those employees whose name has character 'S ' in it

```
select *
from emp
where ename not like '%S%';
```

Function:

It is block of code/set of instructions which performs a specific task is known as Function.

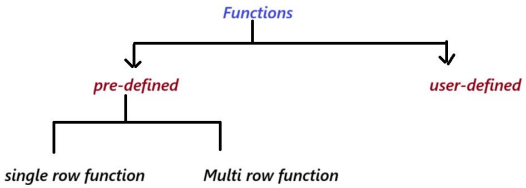
Types of functions
1. Inbuilt/predefined function
2. User defined function

1. Predefined/inbuilt function:

These are the function which are developed by the developer of software. these are the functions available in-built on the specified software and if we want to use those functions it has be called.

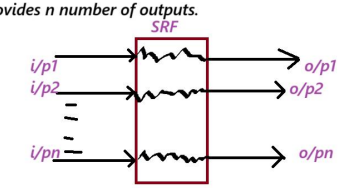
2. Userdefined functions:

These are the type of function which are derived from the users based on their requirements and these functions are known as User defined function.



1. Single Row Function.

These are the function which takes n number of inputs and provides n number of outputs.



--> In SRF for each and every input their is respective o/p is available
--> IN SRF number of input will equals to number of outputs
--> we can pass single Row function in where clause
--> SRF always executes Row By Row.
---> Along with the SRF we can pass any other columns or expression.

Types of MRF are as follows
1. MAX()
2.MIN()
3. SUM()
4.AVG()
5. COUNT()

---Multi Row function has single output for all the inputs
---> Multi Row Function executes in Group
---> In MRF no.of inputs are not equals to no.of outputs.
---> We can not pass MRF in where clause
---> along with the MRF we can not pass any other column names or expression

example:

length():

It is a type of Single row function which is used to provide/return number characters/digits available in the given value.

Synatx:

length(value)

---waqtd name and number of character available on each and every employees name

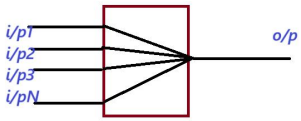
```
select ename,length(ename)
from emp;
```

---waqtd details of the employees who are having exactly 5 characters in there name

```
select  *
from emp
where length(ename)=5;
```

2. Multi Row Function:

these are the function which takes number of inputs at a time combines and executes then provides single output.



MAX():

It is a function which is used to find the maximum value from the given column.

MIN():

It is a function which is used to obtain the Minimum value from the given column.

SUM():

It is used to find /obtain summation of all the values in the given column

AVG():

It is used to obtain the average value of all the values in the given column

COUNT():

It is used to obtain the number of records available in the given column/table.

The Rules of Multi Row Function.

1. Always multi-row function accepts single value as an argument.

```
max(sal,comm)--->it is error
max(sal)---> it is correct
```

2. Along with the multi Row function we are not suppose to pass any other Column_Name or expression but we can other Multi Row Function

```
select Max(sal),ename  ----> it is error
select Max(sal),min(sal),sum(sal)---->it acceptable
```

3. We can not pass MRF in where clause.

```
select *
from emp
where max(sal)>1000;---> it is error
```

4. MRF always ignores the null values on the given column

```
Sum(sal), Max(comm), count(comm)
```

5. Count is the only MRF which accepts * as an argument

```
count(*)---> it is correct
sum(*),avg(*),max(*),min(*)---->error
```

6. Sum() and Avg() are the only MRF which accepts only numerical data type

```
sum(ename),avg(job),sum(hiredate)
```

---WAQTD total salary required pay for all the employee

```
select sum(sal)
from emp;
```

---waqtd average of commission needed to pay for the employees

```
select avg(comm)
from emp;
```

---WAQTD minimum employees name from emp table

```
select min(ename)
from emp;
```

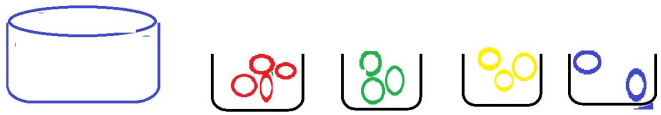
---waqtd maximum hiredate from emp table

```
select max(hiredate)
from emp;
```

---waqtd number of employees who has reporting manager

```
select count(mgr)
from emp;
```

Group by Clause:



It is a type of clause which is used to group the records.

Syntax:

```
select Group_function/groub_by_expression
from Table_Name
[Where <filter_condition>]
group_by column_name/Expression;
```

order of writing the query using group by clause

1. select
2. From
3. WHere
4. group_by

order of execution of group by clause

1. From
2. Where
3. Group by
4. Select

when the given question has each key word in it then we must have use Group by clause in the query based the column on which each key word is mentioned.

---waqtd number of employees on each dept

```
select count(*)
from emp
group  by deptno;
```

empno	ename	sal	deptno
1	Manish	1000	20
2	Puga	1200	10
3	Arpita	1500	30
4	Vidya	2000	10
5	Chandan	2500	30
6	Rishab	1800	10
7	Jemima	2000	20

o/p group by clause

o/p select

1 7	Manish Jemima	1000 2000	20 20	2
2 4 6	Puga Vidya Rishab	1200 2000 1800	10 10 10	3
3 5	Arpita Chandan	1500 2500	30 30	2

Note:

---> group by used to group the records

---> group by clause either executes after from or where based on the where clause available on the query

--->group by clause always executes row by row

---> after execution of group by clause we get groups

---> if any clause that executes after group by clause those always executes Group by Group

Group by Expression:

any column_name/Expression used in the group by clause if those column_name/expression used in the select clause then those called as Group_by_expression.

---waqtd maximum salary on each job
```
select max(sal),job
from emp
group by job;
```

---waqtd sum of sal on each dept in which sal should be greater than 1000
```
select sum(sal),deptno
from emp
where sal>1000
group by deptno;
```

---waqtd sum of sal on each dept in which sal should be greater than 1000
```
select sum(sal),deptno
from emp
where sal>1000
group by deptno;
```

---waqtd average salary on each job except those who are working as clerk
```
select avg(sal),job
from emp
where job!='CLERK'
GROUP BY JOB;
```

HAVING CLAUSE:

It is a type of clause which is used to filter the groups

Syntax:
```
select group_function/group_by_expression
  from Table_Name
  [where <filter_condition>
  group by column_name/Expression
 having  <group_filter_condition>;
```

order of writing the query
1. select
2. from
3. where
4. group by
5. having

order of executing the query
1. From
2. Where [Row by Row]
3. Group by [Row by Row]
4. Having [Group by Group]
5. select [Group by Group]

Group Filter condition:

It is a condition used in a having clause in order to filter the groups.

Syntax:

group_by_expression/MRF operator value

Note:
- ---> Having clause is used to filter the groups
- ---> Having clause is dependent on Group by clause
- ----> Having clause will be executed after group by clause
- ----> Having clause always executes in group by group
- ---->Having clause accepts Multi Row Function.

When to go for using having clause in the query?
--> whenever the condition is given on using MRF or group by expression then we can use Having clause.
--> Whenever the question has at least, at most key word then we use having clause

assignment:

write the difference between Where clause and having clause.

---WAQTD maximum salary on each dept in which maximum salary is greater than 3000
```
select max(sal),deptno
from emp
group by deptno
having max(sal)>3000;
```

---waqtd minimum salary on each job except those who earn the salary greater than 2000 and except salesman
```
SELECT MIN(SAL),JOB
FROM EMP
WHERE NOT SAL>2000
GROUP BY JOB
HAVING JOB!='SALESMAN';
```

OR

```
select min(sal),job
from emp
where sal<=2000 and job!='SALESMAN'
GROUP BY JOB;
```

---WAQTD number of employees on each job in which at least their should be 3 employees working in it
```
select count(*),job
 from emp
 group by job
  having count(*)>=3;
```

---waqtd number of employees on each dept at which their should be at most 4 employees working.
```
select count(*),deptno
from emp
 group by deptno
 having count(*)<=4;
```

---waqtd duplicate salary on emp table
```
select sal
 from emp
 group by sal
 having count(sal)>1;
```

---waqtd repeated hiredate on emp table
```
select hiredate
from emp
group by hiredate
having count(hiredate)>1;
```

Order by clause:

It is a type of clause which is used to arrange the records/sort the records in a particular order it might be ascending/descending.

Syntax:

```
select column_name/Expression
from Table_Name
order by Column_Name [asc]/desc;
```

order of writing is

1. Select
2. From
3. Order by

Order of execution:

1. From
2. Select
3. Order by

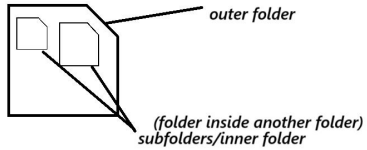
--> Order by clause should be written as last statement in the query
--> Order by clause is the only clause which executes after the execution of Select.
--> By default order by clause arranges the records in ascending order either we mention asc or not.

```
---WAQTD details of the employees at which records are arranges based on salary in an ascending order
  select *
   from emp
   order by sal;
 ---WAQTD different jobs in Descending order
 select distinct job
 from emp
 order by job desc;
SQL> ---waqtd name,job salary and hiredate employees and then arrange the records based on their hiredate
select ename,job,sal,hiredate
from emp
order by hiredate

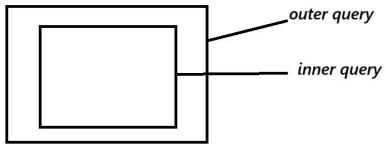
---waqtd names of the employees who salary is greater Allen
  select ename
  from emp
   where sal>'ALLEN';
```

this above query can be solved using Subquery.

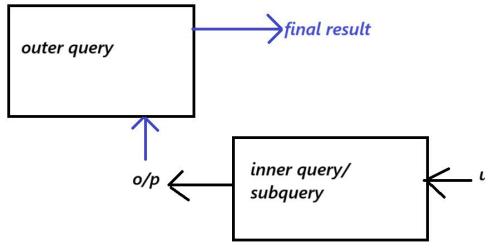
SUBQUERY:



A query written inside another query is known as Subquery.
--> Subquery is also known as Inner query



Working Principle of Subquery:



---> In subquery we have 2 queries one is inner query and the other is outer query
---> Always Inner query executes first by taking the input from the user
---> Inner query executes and provides the output.
---> The output of the inner query is given as an input to the outer query
---> Outer query executes and provides the result
--->Always outer query is responsible to get the final result.

when to go for subquery?

There are 2 case to use subquery.

case1:

Whenever there is unknown in the query

case 2:

whenever the data to selected and condition must be executed are on two different table.

CASE1:

WHENEVER THERE IS UNKNOWN IN THE QUESTION/QUERY WE USE SUBQUERY.

```
---waqtd names of the employees who are getting the salary greater than ALLEN.
  select ename
  from emp
   where sal> (select sal
               from emp
                where ename='ALLEN');
    ---WAQTD details of the employees who are working in the same designation as Miller
    select *
    from emp
    where job =(select job
            from emp
             where ename='MILLER');
                 ---WAQTD name,sal,hiredate of the employees who are hired before ADAMS
                  select ename,sal,hiredate
                  from emp
                  where hiredate<(select hiredate
                           from emp
                            where ename='ADAMS');
```

Case2:

Whenever data to be selected and condition should be executed are on two different table.

```
-WAQTD dname of Scott.
 SELECT DNAME
 FROM DEPT
 WHERE DEPTNO=(SELECT DEPTNO
          FROM EMP
           WHERE ENAME='SCOTT');
---waqtd DETAILS OF THE EMPLOYEES WHO ARE WORKING IN CHICAGO LOCATION
SELECT *
FROM EMP
WHERE DEPTNO =(SELECT DEPTNO
          FROM DEPT
          WHERE LOC='CHICAGO');
```

Queries on Subquery with the combination case1 and Case2:

---WAQTD NAMES OF THE EMPLOYEES WHO ARE HIRED AFTER CLARK AND THEY MUST BE WORKING IN RESEARCH DEPT

```
SELECT ENAME
FROM EMP
WHERE HIREDATE>(SELECT HIREDATE
                FROM EMP
                WHERE ENAME='CLARK') AND DEPTNO=(SELECT DEPTNO
                                                 FROM DEPT
                                                 WHERE  DNAME='RESEARCH');
```

SQL> ---WAQTD DETAILS OF THE EMPLOYEES WHOSE ANNUAL SALARY IS GREATER THAN jAMES AND MUST WORKING IN DALLAS LOCATION

```
SELECT *
FROM EMP
WHERE SAL*12>(SELECT SAL*12
FROM EMP
WHERE ENAME='JAMES') AND DEPTNO=(SELECT DEPTNO
                FROM DEPT
                WHERE LOC='DALLAS');
```

Finding the value of Max and Min

---WAQTD name, salary of the employee who is getting maximum salary

```
 select ename,sal
 from emp
 where sal=( select max(sal)
             from emp);
```

---WAQTD name and job of employee who is getting minimum salary

```
select ename,job
from emp
where sal=(select min(sal)
          from emp);
```

----waqtd details of the employees who hired last

```
select *
from emp
where hiredate=(select max(hiredate)
              from emp);
```

Types of Subquery:

1. Single Row Subquery
2.Multi Row Subquery

1. Single Row Subquery:

It is a type of subquery which returns exactly one records/row as an output then such kind of subqueries are known as Single Row Subquery.

2. Multi Row Subquery:

It is a type of subquery which is returns more than one value as an output then such kind of subqueries are known as multi Row subquery.

---> whenever a inner query /subquery returns more than one value as an output then in the outer query to accept the multiple input we can not use Normal operator so we use Special operator.

If we don't use special operator then query gives an error say "Single Row Subquery Returns More than one Row".

---WAQTD names of the employees who working the same dept as allen and scott

```
select ename
from emp
where deptno=(select deptno
              from emp
              where ename in ('ALLEN','SCOTT'));---> thi query returns error
```

so we use below query by replacing = with IN operator.

```
select ename
from emp
where deptno IN (select deptno
              from emp
              where ename in ('ALLEN','SCOTT'));
```

SQL> ---waqtd name and salary of the employees who are getting the salary more than the employees working as clerk

```
SQL> select ename,sal
  2  from emp
  3  where sal>(select sal
  4             from emp
  5             where job='CLERK');
where sal>(select sal
          *
ERROR at line 3:
ORA-01427: single-row subquery returns more than one row
```

> operator in the outer query can not accept multiple values at RHS so we must use some other special operators Known as ALL and ANY operator

ALL and ANY operator:

These are the operators must be used along with the Relational Operator

```
>ALL                >ANY
<ALL                <ANY
>=ALL               >=ANY
<=ALL               <=ANY
=ALL                =ANY
!=ALL               !=ANY
```

ALL operator:

It is a type of special operator which returns true when condition is true for all the values returned by the subquery

---> This operator is similar to AND operator

---> whenever the question has ALL or no key word of ALL and ANY we must have to use ALL operator

ANY Operator:

It is a type of Special operator which returns true when any one value is true for the value returned by subquery and it returns false when no values are true.

--->This operator is similar to OR operator

--->When the question has ANY keyword in it then only use ANY operator in the query or else go for ALL operator.

---WAQTD name and salary of the employees whose salary is greater than the employees working as Clerk

```
select ename,sal
from emp
   where sal>ALL(select sal
                 from emp
                 where job='CLERK');
```

SQL> ---waqtd NAMES OF THE EMPLOYEE AND THEIR HIREDATE AT WHICH EMPLOYEES HIRED AFTER ANY EMPLOYEES WORKING AS A SALESMAN

```
SQL> SELECT ENAME,HIREDATE
  2  FROM EMP
  3  WHERE HIREDATE>any(SELECT HIREDATE
  4                FROM EMP
  5                WHERE JOB='SALESMAN');
```

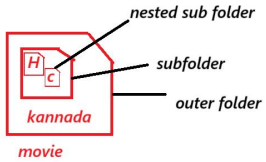
--waqtd names of the employees and their hiredate whose hiredate after all the employees working as analyst.

```
select ename
from emp
where hiredate>ALL(select hiredate
                   from emp
                   where job='ANALYST');
```

SQL> ---waqtd name and job of the employees whose job is same as any of the employees working in deptno 30

```
SQL> SELECT ENAME,JOB
  2  FROM EMP
  3  WHERE JOB=anY(SELECT JOB
  4             FROM EMP
  5             WHERE DEPTNO=30);
```

Nested subquery:

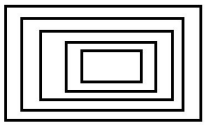


subquery---> query inside another query is known as subquery

Nested subquery:

Subquery inside another subquery is known as Nested subquery.

--> we can nest up to 255 subqueries.



---waqtd names of the employees whose salary is greater than the employees working in sales dept.

```
select ename
from emp
where sal>all  (
        SELECT SAL
        FROM EMP
        where deptno= (select deptno
                       from DEPT
                       where dname='SALES'));
```

---WAQTD DNAME AND LOCATION OF THE EMPLOYEES WHOSE JOB IS SAME AS THE JOB OF ALLEN

```
SELECT DNAME,LOC
FROM DEPT
WHERE DEPTNO IN (SELECT DEPTNO
                 FROM EMP
                 WHER JOB=(SELECT JOB
                           FROM EMP
                           WHERE ENAME='ALLEN'));
```

---WAQTD DETAILS OF THE EMPLOYEES WHO ARE HIRED AFTER ALL THE EMPLOYEES WORKING IN ACCOUNTING DEPT

```
SELECT *
FROM EMP
WHERE HIREDATE>ALL(SELECT HIREDATE
                   FROM EMP
                   WHERE DEPTNO=(SELECT DEPTNO
                                 FROM DEPT
                                 WHERE DNAME='ACCOUNTING'));
```

---WAQTD DNAME OF THE EMPLOYEE WHO IS GETTING MAXIMUM SALARY .

```
SELECT DNAME
FROM DEPT
WHERE DEPTNO=(SELECT DEPTNO
              FROM EMP
              WHERE SAL =(SELECT MAX(SAL)
                          FROM EMP));
```

--WAQTD DEPT DETAILS OF THE EMPLOYEE WHO HIRED FIRST.

```
SELECT *
FROM DEPT
WHERE DEPTNO IN (SELECT DEPTNO
                 FROM EMP
                 WHERE HIREDATE =(SELECT
MIN(HIREDATE)
                                  FROM EMP));
```

FINDING NTH MAXIMUM AND NTH MINIMUM SALARY FROM EMP TABLE.

FINIDNG NTH MAXIMUM SALARY:

SAL
1200
1000
5000
2000
3000
2500
1700

1200
1000
2000
2500
1700

1ST MAXIMUM SAL:

```
SELECT MAX(SAL)
FROM EMP; ---> 5000
```

2ND MAXIMUM SAL

```
SELECT MAX(SAL)
FROM EMP         5000
WHERE SAL<(SELECT MAX(SAL)
           FROM EMP);--->3000
```

3RD MAXIMUM SAL

```
SELECT MAX(SAL)
FROM EMP
WHERE SAL<(SELECT MAX(SAL)
           FROM EMP
           WHERE SAL<(SELECT MAX(SAL)
                      FROM EMP)); --->2500
```

1ST MAX---> 0 SUBQUERY
2ND MAX--->1 SUBQUERY
3RD MAX---->2 SUBQUERY

--
100 MAX----->99 SUB
NTH MAX ----> N-1 SUB
256 MAX---->255 SUB
257 MAX----> WE CAN NOT NEST MORE THAN 255 SUBQUERY

---WAQTD 4TH MAXIMUM SALARY FROM EMP

```
SELECT MAX(SAL)
FROM EMP
WHERE SAL<(SELECT MAX(SAL)
           FROM EMP
           WHERE SAL<(SELECT MAX(SAL)
                      FROM EMP
                      WHERE SAL<(SELECT MAX(SAL)
                                 FROM EMP)));
```

```
---watqd 5th maximum salary from emp table
select max(sal)
from emp
where sal<(select max(sal)
                from emp
                where sal<(select max(sal)
                                from emp
                                where sal<(select max(sal)
                                                from emp
                                                where sal<(select max(sal)
                                                                from emp)))));
```

Finding NTH MINIMUM SALARAY

```
---waqtd first minimum salary from emp;
   select min(sal)
   from emp;

---waqtd second minimum salary from emp
  select min(sal)
  from emp
  where sal>(select min(sal)
               from emp);

---WAQTD third minimum salary from emp
   select min(sal)
    from emp
   where sal>(select min(sal)
                from emp
                  where sal>(select min(sal)
                               from emp));

1st minimum salary--> 0 subquery
2nd minimum salary--->1 subquery
3rd minimum salary--->2 subquery
Nth minimum salary--->N-1 Subquery
256 minimum salary--->255 subquery

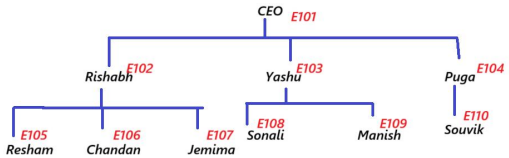
---WAQTD details of the employees who is getting 2nd minimum salary
 select *
from emp
where sal=(select min(sal)
from emp
where sal> (select min(sal)
               from emp));

 ---waqtd name and salary of the employees getting 3rd maximum salary
 select ename,sal
 from emp
 where sal=(select max(sal)
          from emp
          where sal<(select max(sal)
                   from emp
                   where sal<(select max(sal)
                            from emp)));
```

```
---waqtd dname of the employees who is getting 2nd minimum salary
select dname
from dept
where deptno=( select deptno
from emp
where sal=(select min(sal)
from emp
where sal>(select min(sal)
    from emp)));

---waqtd dept details of the employees who is getting 3rd maximum sal
 select *
  from dept
   where deptno=(select deptno
  from emp
   where sal=(select max(sal)
  from emp
   where sal<( select max(sal)
  from emp
    where sal<( select max(sal)
    from emp)));
```

Employees manager Relationship:



EMPNO	ENAME	MGR
E101	CEO	
E102	RISHABH	E101
E103	YASHU	E101
E104	PUGA	E101
E105	RESHAM	E102
E106	CHANDAN	E102
E107	JEMIMA	E102
E108	SONALI	E103
E109	MANISH	E103
E110	SOUVIK	E104

In employee manager relationship there are 2 cases
case 1: Identifying the manager

case2: Identifying the employees

case1:

Identifying the managers.

```
---WAQTD managers name of Puga

 select ename
 from emp
 where empno=(select mgr
 from emp
 where ename='PUGA');

 ---waqtd manager details of MANISH
 SELECT *
 FROM EMP
 WHERE EMPNO=(SELECT MGR
                    FROM EMP
                    WHERE ENAME='MANISH');

 ---WAQTD MANAGER'S MANAGER NAME OF SONALI
    SELECT ENAME
    FROM EMP                E101
    WHERE EMPNO=(SELECT MGR
                      FROM EMP      E103
                  WHERE EMPNO=(SELECT MGR
                                    FROM EMP
                                WHERE ENAME='SONALI'));


   ---WAQTD MANAGER DETAILS OF SMITH
    SELECT *
    FROM EMP
    WHERE EMPNO=  (SELECT MGR
     FROM EMP
     WHERE ENAME='SMITH');

   ---WAQTD DNAME OF SCOTT'S MANAGER
   SELECT DNAME
   FROM DEPT
   WHERE DEPTNO=(SELECT DEPTNO
   FROM EMP
   WHERE EMPNO=(SELECT MGR
            FROM EMP
             WHERE ENAME='SCOTT'));

   ---WAQTD MANAGER'S MANAGER NAME OF CLARK
    SELECT ENAME
   FROM EMP
   WHERE EMPNO=(SELECT MGR
                   FROM EMP
                    WHERE EMPNO=(SELECT MGR
                                    FROM EMP
                                     ENAME='CLARK'));
```

```
--WAQTD MANAGER'S MANAGER DETAILS OF FORD
 SELECT *
 FROM EMP
 WHERE EMPNO=(SELECT MGR
          FROM EMP
          WHERE EMPNO=(SELECT MGR
                   FROM EMP
                   WHERE ENAME='FORD'));
```

CASE2 :

IDENTIFYING THE EMPLOYEES.

```
---WAQTD NAMES OF THE EMPLOYEES REPORTING TO CEO.
   SELECT ENAME
  FROM EMP
  WHERE MGR= (SELECT EMPNO
                   FROM EMP
                   WHERE ENAME='CEO');

---WAQTD DETAILS OF THE EMPLOYEES WHO ARE REPORTING JONES
  SELECT *
  FROM EMP
   WHERE MGR=(SELECT  EMPNO
                 FROM EMP
                 WHERE ENAME='JONES');
```

finding employees---> find empno in subquery
finding managers----> find mgr in subquery

----waqtd manager name of Miller

```
select ename
from emp
where empno=(select mgr
                from emp
                where ename='MILLER');
```

---WAQTD DNAME OF JONES MANAGER

```
SELECT DNAME
FROM DEPT
WHERE DEPTNO=(SELECT DEPTNO
          FROM EMP
          WHERE EMPNO=(SELECT MGR
                    FROM EMP
                    WHERE ENAME='JONES'));
```

---WAQTD MANAGER'S DETAILS OF THE EMPLOYEES WHO ARE WORKING AS CLERK

```
SELECT *
FROM EMP
WHERE EMPNO in (SELECT MGR
          FROM EMP
          WHERE JOB='CLERK');
```

---WAQTD MANAGER NAME AND THE SALARY OF THE EMPLOYEES WHO ARE WORKING IN DALLAS LOC

```
SELECT ENAME,SAL
FROM EMP
WHERE EMPNO IN(SELECT MGR
          FROM EMP
          WHERE DEPTNO=(SELECT DEPTNO
                    FROM DEPT
                    WHERE LOC='DALLAS'));
```

---WAQTD NUMBER OF EMPLOYEES REPROTING TO KING

```
SELECT COUNT(*)
FROM EMP
WHERE MGR=(SELECT EMPNO
        FROM EMP
        WHERE ENAME='KING');
```

---waqtd maximum salary of the employees who are reporting to blake

```
select max(sal)
from emp
where mgr=(select empno
                from emp
                where ename='BLAKE');
```

---WAQTD NAME AND SALARY OF THE EMPLOYEES WHO ARE REPORTING TO MARTIN MANAGER.

```
SELECT ENAME,SAL
FROM EMP
WHERE MGR=(SELECT MGR
        FROM EMP
        WHERE ENAME='MARTIN');
```

---WAQTD NAME AND DNAME OF THE EMPLOYEES.

To solve this query we use Joins

JOINS:

join means combine/merge

why we need to join 2 tables.

In order to retrieve the records from multiple tables we need to join two tables.

Joins:

The Process of Retrieving the multiple records from two different table simultaneously.

Types of joins:

1. Cartesian Join/cross join
2. Inner join/equi join
3. Natural Join
4. Outer join
   i. left outer join
   ii. Right outer join
   iii. Full outer join
5. Self join

1. Cross join/Cartesian Join:

It is a type of join at which each and every record from 1 table is combined with each and every record of table2.

---> the number of records in the output of cross join will equal to the product of records in table1 and table2.

syntax:

ANSI:

```
select */Column_Name
from table_name1 cross join table_name2;
```

Oracle:

```
select */Column_Name
from table_name1,table_name2;
```

empno	ename	deptno
1	Ajit	20
2	Sujit	10
3	Amit	20
4	Poojit	30

deptno	dname	loc
10	account	Hyderabad
20	sales	Chennai
30	operation	Mumbai
40	research	Pune

---waqtd details employees and dept

```
select *
from emp cross join dept;
```

```
select *
from emp,dept;
```

empno	ename	deptno	deptno	dname	loc
1	Ajit	20	10	account	Hyderabad
1	Ajit	20	20	sales	Chennai
1	Ajit	20	30	operation	Mumbai
1	Ajit	20	40	research	Pune
2	Sujit	10	10	account	Hyderabad
2	Sujit	10	20	sales	Chennai
2	Sujit	10	30	operation	Mumbai
2	Sujit	10	40	research	Pune
3	Amit	20	10	account	Hyderabad
3	Amit	20	20	sales	Chennai
3	Amit	20	30	operation	Mumbai
3	Amit	20	40	research	Pune
4	Poojit	30	10	account	Hyderabad
4	Poojit	30	20	sales	Chennai
4	Poojit	30	30	operation	Mumbai
4	Poojit	30	40	research	Pune

Note:

cross join gives more number of mismatch record compared to matching records.

---> It is the drawback of cross join.

----waqtd name and dname of the employees

```
select ename,dname
from emp,dept;
```

2. INNER JOIN/EQUI JOIN:

It is a type of join which is used to retrieve multiple records from 2 tables simultaneously and get only matching records between those table.

---> to get the matching records between 2 tables we need a relation between 2 tables.

Syntax:

1. ANSI:

```
select */Column_Name
 from table_name1 inner join table_name2
  on < inner join condition>;
```

2. Oracle:

```
select */Column_Name
 from table_name1,table_name2
 where <inner join condition>;
```

inner join condition:

It is condition used to get only matching records from two tables.

syntax:

```
Table_Name1.common_column=table_name2.common_column;
```

empno	ename	deptno
1	Manish	30
2	Souvik	10
3	Chandan	20
4	Jemima	10
5	Vidya	30

deptno	dname	loc
10	account	delhi
20	sales	Mumbai
30	research	Chennai
40	operation	Pune

```
---waqtd details of employees and dept
 select *
 from emp inner join dept
  on   emp.deptno=dept.deptno;

    or
select *
from emp,dept
where emp.deptno=dept.deptno;
```

empno	ename	deptno	deptno	dname	loc
1	Manish	30	30	research	Chennai
2	Souvik	10	10	account	delhi
3	Chandan	20	20	sales	Mumbai
4	Jemima	10	10	account	delhi
5	Vidya	30	30	research	Chennai

```
----waqtd name,salary and location of the employees who are having character a in their dname
select ename,sal,loc
from emp inner join dept
on emp.deptno=dept.deptno and dname like '%A%';

select ename,sal,loc
from emp inner join dept
on emp.deptno=dept.deptno
where dname like '%A%';

or

 select ename,sal,loc
 from emp,dept
 where emp.deptno=dept.deptno and dname like '%A%';

---waqtd ename,job and dname of the employee along with their annual salary at which employees are getting the salary more than Allen
select ename,job,dname
from emp,dept
where emp.deptno=dept.deptno and sal>(select sal
                                      from emp
                                       where ename='ALLEN');

SQL> ----WAQTD DETAILS OF THE EMPLOYEES ALONG WITH DISPLAY THE LOCATION OF THE EMPLOYEES WHO ARE HAV
ING CHARACTER 'E' IN THEIR NAME AND MUST BE WORKING AS SALESMAN
SQL> SELECT EMP.*,LOC
  2  FROM EMP,DEPT
  3  WHERE EMP.DEPTNO=DEPT.DEPTNO AND ENAME LIKE '%E%' AND JOB='SALESMAN';

---WAQTD NAMES OF THE EMPLOYEES WORKING IN ACCOUNTING DEPT
SELECT ENAME
FROM EMP,DEPT
WHERE EMP.DEPTNO=DEPT.DEPTNO AND DNAME='ACCOUNTING';

SQL> ---WAQTD DETAILS OF THE EMPLOYEES WHO ARE WORKING IN THE SAME DEPT AS SCOTT AND MUST BE WORKING IN THE SAME JOB AS MILLER
SQL> SELECT EMP.*
  2  FROM EMP,DEPT
  3  WHERE EMP.DEPTNO=DEPT.DEPTNO AND EMP.DEPTNO=(SELECT DEPTNO
  4                                FROM EMP
  5                                WHERE ENAME='SCOTT') AND JOB=(SELECT JOB
  6                                                FROM EMP
  7                                                WHERE ENAME='MILLER');
```

NATURAL JOIN:

It is a type of a join which is used whenever we are un aware the relation between 2 tables then we use this Natural join.

Natural join works in 2 different ways:

-->if their is a relation between 2 tables it gives the output similar to inner join

--> if there is no relation between 2 tables it gives the output similar to cross join.

Syntax:

ANSI:

```
select */Column_Name
from table_name1 Natural Join Table_Name2;

---waqtd details of employees and dept table details.
select *
from emp natural join dept;
```

the output is similar to inner join and here natural takes only unique columns from the table.

```
---waqtd details employees and salgrade.
select *
from emp natural join salgrade;
```

the output is similar to Cross join because their is no relation between emp and salgrade table.

Outer join:

It is a type of join which is used to get unmatched records along with the matching records.

Types of outer join are as follows:

1. Left Outer Join
2. Right Outer Join
3. Full Outer join

1. Left Outer Join:

It is a type of join which is used to get unmatched records from left table and matching records from both the tables.

Syntax:

ANSI:

```
select */Column_Name
from table_name1 Left [outer] join table_name2
on table_name1.common_column=table_name2.common_column;
```

Oracle:

```
select */Column_Name
 from table_name1 ,table_name2
where table_name1.common_column=table_name2.common_column(+);
```

empno	ename	deptno
1	Yashu	10
2	Reshma	
3	Souvik	20
4	Anjali	
5	Manish	30

deptno	dname	loc
10	Account	Delhi
20	Sales	Mumbai
30	Research	Pune
40	Operation	Chennai

----waqtd details of employees and dept along with that display those employees who are not assigned to any dept.

```
select *
from emp left join dept
on emp.deptno=dept.deptno;
```

OR

```
select *
from emp,dept
where emp.deptno=dept.deptno(+);
```

empno	ename	deptno	deptno	dname	loc
1	Yashu	10	10	Account	Delhi
3	Souvik	20	20	Sales	Mumbai
5	Manish	30	30	Research	Pune
2	Reshma	null	null	null	null
4	Anjali	null	null	null	null

Right outer join:

It is a type of outer join which is used to get unmatched record from right table along with that display matching records from both table.

syntax:

ANSI:

```
select *
from table_name1 right [outer] join table_name2
on table_name1.common_column=table_name2.common_column;
```

oracle:

```
select *
from table_name1,table_name2
where table_name1.common_column(+)=table_name2.common_column;
```

empno	ename	deptno
1	Yashu	10
2	Reshma	
3	Souvik	20
4	Anjali	
5	Manish	30

deptno	dname	loc
10	Account	Delhi
20	Sales	Mumbai
30	Research	Pune
40	Operation	Chennai

----WAQTD details of employees and dept details along with that display the dept at which no employees are working

```
select *
from emp right join dept
on emp.deptno=dept.deptno;
```

```
select *
from emp,dept
where emp.deptno(+)=dept.deptno;
```

empno	ename	deptno	deptno	dname	loc
1	Yashu	10	10	Account	Delhi
3	Souvik	20	20	Sales	Mumbai
5	Manish	30	30	Research	Pune
null	null	null	40	operation	Chennai

Full Outer join:

It is a type of outer join which is used to get unmatched records from both the table along with displays matching records also from both the tables.

Syntax:

ANSI:

```
select */Column_Name
from table_name1 Full [outer] join table_name2
on table_name1.common_column=table_name2.common_column.
```

---waqtd details of employees and dept table along with that display employees whoa re not assigned to any dept and display those dept at which no employees are working.

```
select *
from emp full join dept
on emp.deptno=dept.deptno;
```

empno	ename	deptno
1	Yashu	10
2	Reshma	
3	Souvik	20
4	Anjali	
5	Manish	30

deptno	dname	loc
10	Account	Delhi
20	Sales	Mumbai
30	Research	Pune
40	Operation	Chennai

empno	ename	deptno	deptno	dname	loc
1	Yashu	10	10	Account	Delhi
3	Souvik	20	20	Sales	Mumbai
5	Manish	30	30	Research	Pune
2	Reshma	null	null	null	null
4	Anjali	null	null	null	null
null	null	null	40	operation	Chennai

1. SELF JOIN:

It is a type of join which is used to retrieve the multiple records from the same table simultaneously .

--> joining of 2 same tables are known as self join.

syntax:

ANSI:

```
select */Column_Name
 from table_name Join Table_Name
  on <self_join_condition>;
```

Oracle:

```
select */Column_Name
 from Table_Name,table_name
   where <self_join_condition>;
```

self join condition:

It is a condition which is used to join two same tables.

----waqtd names of the employees and their respective managers name.

```
select e1.ename,e2.ename
from emp e1,emp  e2
where e1.mgr=e2.empno;
```

---waqtd names ,salary employees and managers respectively

```
select e1.ename,e1.sal,e2.ename,e2.sal
from emp e1,emp e2
where e1.mgr=e2.empno;
```

SQL> ---waqtd name and job of the employees and managers name at which employees are working in deptno 30

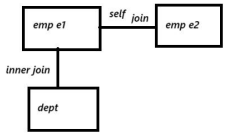
```
SQL> select e1.ename,e1.job,e2.ename
  2  from emp e1,emp e2
  3  where e1.mgr=e2.empno and e1.deptno=30;
```

---waqtd details of manager along with display job and name of employees

```
select e2.*,e1.job,e1.ename
from emp e1,emp e2
where e1.mgr=e2.empno;
```

```
---waqtd name of the employees names of the manager and names of manager's manager
  select e1.ename,e2.ename,e3.ename
  from emp e1,emp e2,emp e3
  where e1.mgr=e2.empno and e2.mgr=e3.empno;

---waqtd name and job of employees and managers along with that display the dname of employees.
```




```
select e1.ename,e1.job,e2.ename,e2.job,dname
from dept,emp e1,emp e2
where e1.mgr=e2.empno and e1.deptno=dept.deptno;

 ---waqtd name of the employees and manager then diplay the location employees and managers
 select e1.ename,e2.ename,d1.loc,d2.loc
 from emp e1,emp e2,dept d1,dept d2
 where e1.mgr=e2.empno and d1.deptno=e1.deptno and e2.deptno=d2.deptno;

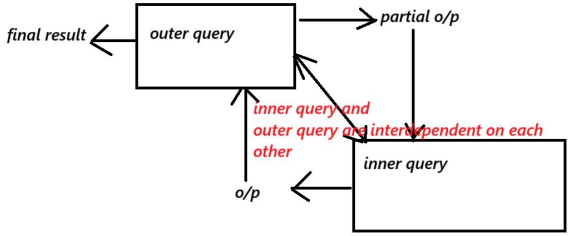
SQL> ---waqtd names of the employees and manager name at which employees and manager are working in same dept
SQL> select e1.ename,e2.ename
  2  from emp e1,emp e2
  3  where e1.mgr=e2.empno and e1.deptno=e2.deptno;

SQL> --waqtd name and salary of employees and manager respectively if employees are getting the salary more than their managers
 select e1.ename,e1.sal,e2.ename,e2.sal
 from emp e1,emp e2
 where e1.mgr=e2.empno and e1.sal>e2.sal;
```

Co-Related Subquery:

It is a type of subquery which works on the principle of subquery and Join concept.

Working Principle of Co-related subquery:




```
--> In co-related subquery always outer query executes first and provides the partial output
--> the partial output is given as an input to the inner query
--> inner query executes and provides the output
---> the output of the inner query is given as an input to the outer query
---> outer executes by taking the input from the inner query and provides the
      final result
---> by this we conclude that outer query and inner query are interdependent on each other

---waqtd dname of the employees at which employees are working
   select d.dname
    from dept d            10,20,30,null
     where d.deptno in (select e.deptno
                        from emp e
                       where d.deptno=e.deptno);

in the above query we are checking the same condition twice so reduce one of the condition
we use A special subquery operators known as EXISTS and NOT EXISTS.

EXISTS:
        It is a type of unary operator which returns for the values returned by subquery
and it returns false for the null value returned by the subquery.

select d.dname
from dept d
where EXISTS(select e.deptno
               from emp e
              where d.deptno=e.deptno);
```

Not EXISTS :

It is a type of subquery unary operator which returns true for the null and it returns false the other than null values returned by subquery

```
---waqtd dname of the employees at which no employees are working
  select d.dname
  from dept d
   where not Exists(select e.deptno
                    from emp e
                     where d.deptno=e.deptno);
```

Finding Maximum and Minimum salary

Syntax for finding Nth maximum salary.

```
select e1.sal
from emp e1
where N-1 in (select count(distinct e2.sal)
              from emp e2
              where e1.sal<e2.sal);

waqtd 3rd maximum salary from emp;
select e1.sal
from emp e1
where 3-1 in (select count(distinct e2.sal)
              from emp 2
               where e1.sal<e2.sal);

 ---waqtd 7th maximum salary from emp
select e1.sal
from emp e1
where 7-1 in (select count(distinct e2.sal)
              from emp e2
               where e1.sal<e2.sal);

---waqtd 3,5, and 7th maximum salary from emp
select e1.sal
from emp e1
where   (select count(distinct e2.sal)
              from emp e2
               where e1.sal<e2.sal) in (3-1,5-1,7-1);
```

Syntax for finding Nth minimum salary

```
select e1.sal
from emp e1
where N-1 in (select count(distinct e2.sal)
              from emp e2
               where e1.sal>e2.sal);

----waqtd 6th minimum salary from emp
select e1.sal
from emp e1
where 6-1 in (select count(distinct e2.sal)
              from emp e2
               where e1.sal>e2.sal);

---waqtd 7,8 and 10 th minimum salary from emp
select e1.sal
from emp e1
where (select count(distinct e2.sal)
              from emp e2
               where e1.sal>e2.sal)in(7-1,8-1,10-1);
```

DDL statement:

Data Definition Language

It is a type of a SQL statement which is used to create the table and deals with the structure of the table.

Types of DDL statements:

1. Create
2. Rename
3. Alter
4. Truncate
5. Drop

1. Create:

Create is used to create the table in the data base.

Syntax:

```
create table Table_Name(column_name1 data_type constraints,
                        column_name2 data_type constraints,
                        -----
                        ----
                        column_nameN data_type constraints
                        );
```

----waqtd create a table called demo with columns like Id, name,phone

```
create table demo(id number(5) primary key,
                  name varchar(15) not null,
                  phone number(10) unique not null check(phone>0) and check(length(phone)=10)
                  );
```

----waqt create a table product with the columns like PID,pname,price of product

```
create table product(pid number(6) primary key,
                     pname varchar(30) not null,
                     price number(7,2) not null check(price>0)
                     );
```

2. RENAME:

It is a type of DDL statement which is used to change the existing Table_Name to new Table_Name.

Syntax:

```
rename existing_table_name to new_table_name;
```

---waqt change the name of demo table to dummy table

```
rename demo to dummy;
```

3. Alter:

It is a type of DDL statement which is used change the structure of the existing table by altering the below once

1. by adding column
2. by dropping column
3. by renaming Column_Name
4. by modifying the data type
5. by modifying the constraints
6. by adding a constraints
7. By adding a foreign key constraint

1. By adding a column:

It is used to add the new column to the existing table.

syntax:

```
alter table Table_Name
 add Column_Name data_type constraints.
```

---waqt add email column to the dummy table

```
alter table dummy
add email varchar(25) unique;
```

--waqtd MFD(manufacture date) column to the Product table

```
alter table product
add MFD date not null;
```

2. By dropping column:

It is used to drop the existing column of a table.

syntax:

```
alter table Table_Name
drop column Column_Name;
```

---waqt drop MFD from the product table.

```
alter table product
drop column MFD;
```

3. By renaming the Column_Name:

It is used to rename the Column_Name of existing table

syntax:

```
alter table Table_Name
rename column exiting_column_name to new_column_name;
```

---waqt alter the phone to mobile in dummy table

```
alter table dummy
rename column phone to mobile;
```

4. Modifying the data type:

It is used to change the data type of existing column of a table.

syntax:

```
alter table Table_Name
modify Column_Name new_data_type;
```

---waqt modify the data type of Id to Char(6) of a dummy table

```
alter table dummy
modify id char(6);
```

5. Modifying the constraints:

It is a used to change the constraint

--> using this we can change either Null to Not null or not null to null.

syntax:

```
alter table Table_Name
modify Column_Name data_type new_constraint [Null/Not null];
```

---waqtd not null constraint to email column of dummy table

```
alter table dummy
modify email number(30) not null;
```

constraint reference name:

These are the unique names for the constraint for the unique identification.

---> It should be unique in the entire data base.

--> using this we can drop a particular constraint of a column.

how to check the constraint available on the given table.

```
select *
from user_constraints
where Table_Name='TABLE_NAME';
```

example:

```
select *
 from user_constraints
 where Table_Name='DUMMY';
```

---waqt drop email column from dummy

```
alter table dummy drop column email;
```

---waqt add email column to dummy table without assign unique constraint

```
alter  table dummy add email varchar(25);
```

assignment:

--waqt create customer table with CID ,cname and Phone

6. By assigning the constraint:

It is used to assign the constraint to the existing column of a table.

syntax:

```
alter table Table_Name
 add constraint con_ref_name constraint_name(Column_Name);
```

---waqt add unique constraint to the email column of Dummy table.

```
alter table Dummy
add constraint Uni_email  Unique(email);
```

7. By dropping the constraint:

It is used to drop the constraint from the particular column of a table.

syntax:

```
alter table Table_Name
 drop constraint constraint_ref_name;
```

---waqt drop unique constraint from the email column of a dummy table

```
alter table dummy
drop constraint uni_email;
```

8. Assigning the Foreign key constraint:

1. after the tables creating
2. during table creating

1. adding a foreign when already the tables are present.

pid	pname	price

cid	cname	phone

Syntax:

```
alter table Table_Name
 add Column_Name data type
 add constraint con_ref_name Foreign key(Column_Name)
  References parent_table_name(Column_Name);
```

---waqt establish the relation between product and customer table

```
alter table customer
add pid number(6)
add constraint FK_pid Foreign key(pid)
references product(pid);
```

2. ADDING FOREIGN KEY DURING TABLE CREATING:

syntax:

```
create table child_table_name(column_name1 data_type constraints,
                              column_name2 data_type constraints,
                              ---
                              Parent_column_name data_type ,
                              constraints con_ref_name Foreign_key(parent_column_name)
                              references parent_table_name(parent_column_name)
                              );
```

---waqt create student and teachers table and establish the connection between them.

creation of parent table known as teachers:

```
create table teachers(tid number(5) primary key,
                      tname varchar(25) not null
                     );
```

creation of child table known as students.

```
create table students(sid number(6) primary key,
                      sname varchar(20) not null,
                      tid number(6),
                      constraints FK_tid foreign key(tid)
                       references teachers(tid)
                       );
```

TRUNCATE:

It is a type of DDL statement which is used to delete the entire records of a table permanently without deleting the table structure.

syntax:

```
truncate table Table_Name;

truncate table demo;
```

demo

id	name
1	vidya
2	seema
3	padma
4	manish

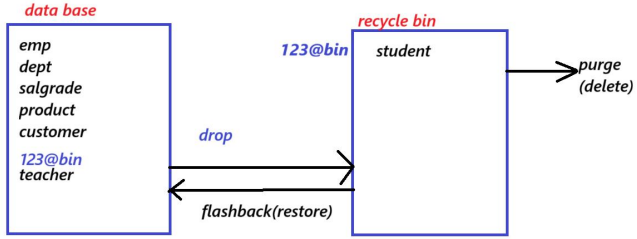
---> we can not truncate the parent table directly.

5. DROP:

It is used to remove or delete the entire table from the data base and it is not permanent.

syntax:

```
drop table Table_Name;
```



command to check the recyclebin

```
select *
from recyclebin;
```

flashback:

It is type of statement which is used take table dropped table back to the data base

Syntax:

```
flashback table Table_Name
 to  before drop;
```

purge:

It is a type of statement which is used delete the table from the recyclebin

syntax:

```
purge table Table_Name;
```

```
SQL> drop table students;

Table dropped.

SQL> select * from tab;

QL> select * from recyclebin;

SQL> flashback table students
  2  to before drop;

Flashback complete.
```

```
SQL> DROP TABLE STUDENTS;

Table dropped.

SQL> SELECT * FROM RECYCLEBIN;

SQL> PURGE TABLE STUDENTS;

Table purged.

SQL> SELECT * FROM RECYCLEBIN;

no rows selected

SQL> SELECT * FROM TAB;
```

DML(data Manipulation Language):

It is a type of SQL statement which is used to insert and manipulate the records on the existing table.

Types of DML statements:

1. INSERT
2. UPDATE
3. DELETE

1. Insert :

It is a type of DML statement which is used to insert the records on the table.

syntax:

1 whenever we know the sequence of column

```
insert into Table_Name values(val1,val2.....valN);
```

2. Whenever we don't know the sequence of columns

```
insert into Table_Name(column_name1,column_name2...column_nameN) values(val1,val2....valN);
```

3. Inserting multiple values using single syntax:

```
insert into Table_Name values(&column_name1,&column_name2,.....,&column_nameN);
  enter value of column_name1:
  enter value of column_name2:
  -----
  enter value of column_nameN:
    /
  enter value of column_name1:
  enter value of column_name2:
  -----
  enter value of column_nameN:
```

```
----waqt insert the records on product table.
SQL> insert into product values(10001,'HP',30000);

1 row created.

SQL> INSERT INTO PRODUCT(PNAME,PID,PRICE)VALUES('LENOVO',10002,15000);

1 row created.

SQL> SELECT *
  2  FROM PRODUCT;

SQL> INSERT INTO PRODUCT VALUES(&PID,'&PNAME',&PRICE);
Enter value for pid: 10003
Enter value for pname: ASUS
Enter value for price: 25000
old   1: INSERT INTO PRODUCT VALUES(&PID,'&PNAME',&PRICE)
new   1: INSERT INTO PRODUCT VALUES(10003,'ASUS',25000)

1 row created.

SQL> /
Enter value for pid: 10004
Enter value for pname: DELL
Enter value for price: 35000
old   1: INSERT INTO PRODUCT VALUES(&PID,'&PNAME',&PRICE)
new   1: INSERT INTO PRODUCT VALUES(10004,'DELL',35000)
1 row created.
```

2. UPDATE:

It is a type of DML statement which is used to update the records available on the table.

Syntax:

```
update Table_Name
 set column_ name1=value,column_name2=value
  where <condition>;
```

```
---waqt update the price of ASUS to 30000
  update product
  set price=30000
  where pname='ASUS';

  ---waqtd update the pname of lenovo to lenovo13
   update product
   set pname='LENOVO13'
  WHERE PNAME='LENOVO';
```

3. DELETE:

It is a type of DML statement which is used to delete the particular record from the table.

syntax:

```
delete from Table_Name
where  <condition>;
```

```
---waqt delete a mac book from product table.
  delete from product
    where pname='MAC BOOK';
```

DDL statements are auto commit(automatic save)
but DML statements are not auto commit.

so to save the records on the table we use TCL statements.

TCL:(TRANSCATION CONTROL LANGUAGE)

Transactions:

These are the operations that are done by the dml statements are known as Transactions.

TCL:

It is a type SQL statement which is used to control the transactions done by the DML statements.

types of TCL statements are:

1. Commit
2. Rollback
3. Savepoint

1. Commit:

It is a type of TCL statement which is used to save the records after each transactions.

syntax:

```
      commit.
---> commit command work on each transaction
      on insert/on update/on delete
```

EXAMPLE:

```
SQL> insert into product values(101,'FANTA',50);

1 row created.

SQL> COMMIT;

Commit complete.
```

ROLLBACK:

It is a type of TCL statement which is used to undo the records till latest /last commit command.

syntax:

```
      Rollback;
```

```
rec1-----
rec2----
commit
rec3---
rec4---
rec5---
rollback
```

```
SQL> insert into product values(101,'FANTA',50);

1 row created.

SQL> COMMIT;

Commit complete.
SQL> insert into product values(&pid,'&pname',&price);
Enter value for pid: 102
Enter value for pname: Good day
Enter value for price: 10000
old   1: insert into product values(&pid,'&pname',&price)
new   1: insert into product values(102,'Good day',10000)

1 row created.

SQL> /
Enter value for pid: 103
Enter value for pname: Pepsi
Enter value for price: 75
old   1: insert into product values(&pid,'&pname',&price)
new   1: insert into product values(103,'Pepsi',75)

1 row created.
```

```
SQL> select * from product;

       PID PNAME                                PRICE
---------- ---------------------------------------- ----------
       101 FANTA                                    50
       102 Good day                              10000
       103 Pepsi                                    75

SQL> rollback;

Rollback complete.

SQL> select * from product;

       PID PNAME                                PRICE
---------- ---------------------------------------- ----------
       101 FANTA                                    50
```

3. Savepoint:

It is a type of TCL statement which is used to provide restoration point on the inserted records.

---> These restoration points help us undo/rollback the certain records.

syntax:

savepoint savepoin_name;

rec1---
rec2----
commit
rec3---- t1(savepoint)
rec4---
rec5--- t2
rec6---
rec7--- t3
rollback to t2



```
SQL> insert into product values(102,'HAPPY HAPPY',30.78);

1 row created.

SQL> SAVEPOINT  T1;

Savepoint created.

SQL> INSERT INTO PRODUCT VALUES(103,'LAYS',20);

1 row created.

SQL> INSERT INTO PRODUCT VALUES(104,'COCO COLA',89.78);

1 row created.

SQL> SAVEPOINT T2;

Savepoint created.

SQL> INSERT INTO PRODUCT VALUES(105,'BOURBAN',50);

1 row created.

SQL> INSERT INTO PRODUCT VALUES(106,'BINGO',30);

1 row created.

SQL> SAVEPOINT T3;

Savepoint created.

SQL> INSERT INTO PRODUCT VALUES(107,'PEPSI',100);

1 row created.

SQL> SELECT * FROM PRODUCT;

       PID PNAME                          PRICE
---------- ------------------------------ ----------
       101 FANTA                          50
       102 HAPPY HAPPY                    30.78
       103 LAYS                           20
       104 COCO COLA                      89.78
       105 BOURBAN                        50
       106 BINGO                          30
       107 PEPSI                          100

7 rows selected.

SQL> ROLLBACK TO T3;

Rollback complete.

SQL> ROLLBACK TO T2;

Rollback complete.

SQL> SELECT * FROM PRODUCT;

       PID PNAME                          PRICE
---------- ------------------------------ ----------
       101 FANTA                          50
       102 HAPPY HAPPY                    30.78
       103 LAYS                           20
       104 COCO COLA                      89.78

SQL> ROLLBACK TO T3;
ROLLBACK TO T3
*
ERROR at line 1:
ORA-01086: savepoint 'T3' never established


SQL> COMMIT;

Commit complete.

SQL> ROLLBACK TO T1;
ROLLBACK TO T1
*
ERROR at line 1:
ORA-01086: savepoint 'T1' never established
```

DCL statement:(Data Control Language)

It is a type of SQL statement which is used to control the data flow between 2 users.

Types of DCL statements:
1. Grant
2. Revoke

GRANT:

It is a type of DCL statement which is used to GRANT the access of a table to other users.

syntax:

```
Grant  SQL_statements
  on Table_Name
 To user_name;
```

REVOKE:

It is a type of DCL statement which is used to take back the access which was granted.

--> the person who grant the permission only that person can take that access back/revoke it back.

syntax:

```
REVOKE SQL_statements
 on Table_Name
  From user_name;
```

```
SQL> show user
USER is "SCOTT"
SQL>
SQL> connect
Enter user-name: hr
Enter password: *****
Connected.
SQL> show user
USER is "HR"
SQL>
SQL> select * from tab;

TNAME                          TABTYPE  CLUSTERID
------------------------------ ------- ----------
REGIONS                        TABLE
COUNTRIES                      TABLE
LOCATIONS                      TABLE
DEPARTMENTS                    TABLE
JOBS                           TABLE
EMPLOYEES                      TABLE
JOB_HISTORY                    TABLE
EMP_DETAILS_VIEW               VIEW
```

```
SQL> connect
Enter user-name: scott
Enter password: *****
Connected.
SQL>
SQL> grant select,insert
  2  on product
  3  to hr;

Grant succeeded.

SQL> connect
Enter user-name: hr
Enter password: *****
Connected.
SQL> select *
  2  from product;
from product
     *
ERROR at line 2:
ORA-00942: table or view does not exist


SQL> select *
  2  from scott.product;

       PID PNAME                          PRICE
---------- ------------------------------ ----------
       101 FANTA                          50
       102 HAPPY HAPPY                    30.78
       103 LAYS                           20
       104 COCO COLA                      89.78

SQL> delete from scott.product
  2  where pid=101;
delete from scott.product
            *
ERROR at line 1:
ORA-01031: insufficient privileges


SQL> insert into scott.product values(105,'GOOD DAY',100);

1 row created.
```

```
SQL> CONNECT
Enter user-name: SCOTT
Enter password: *****
Connected.

SQL> REVOKE SELECT,INSERT
  2  ON PRODUCT
  3  FROM HR;

Revoke succeeded.

SQL> CONNECT
Enter user-name: HR
Enter password: *****
Connected.
SQL>
SQL> SELECT *
  2  FROM SCOTT.PRODUCT;
FROM SCOTT.PRODUCT
     *
ERROR at line 2:
ORA-00942: table or view does not exist


SQL> INSERT INTO SCOTT.PRODUCT VALUES(106,'PEPSI',89);
INSERT INTO SCOTT.PRODUCT VALUES(106,'PEPSI',89)
            *
ERROR at line 1:
ORA-00942: table or view does not exist
```

Normalization:

To understand about the Normalization we need to know about types of attribute and Functional Dependency

Types of attribute:

1. KEY attribute
2. NON Key attribute
3. Prime Key attribute
4. Non Prime key attribute
5. Composite Key attribute
6. Super Key attribute
7. Foreign key attribute.

1. Key attribute:

These are the attributes which accepts unique values are known as Key attributes
---> these are attributes are used to identify the records uniquely

emp: empno,ename,sal,job,hiredate,comm,email,deptno,phone

example :

empno,email,phone

2. Non key attribute:

In a set of attributes other the key attributes remaining attributes are known as Non key attributes.
---> these are the attribute which takes duplicate values.

example: ename,sal,job,hiredate,comm,deptno

3. Prime Key attribute:

In a set of key attribute we choose only one attribute to identify the other records of the table uniquely are known as Prime key attribute.

key attribute: empno,email,phone

example:

empno

4. Non Prime Key attribute:

In a set of key attribute other than the prime key attribute remaining attributes are called as Non prime key attribute

example:

email,phone

5. Composite key attribute:

It is a combination of more than on non key attribute which identify the records uniquely from the table are known as Composite key attribute.

sname,marks, subject,class, course

example:

(sname,marks,course)

6. Super key attribute:

set of all the key attributes are known as Super key attribute.

example:

(empno,email,phone)

7. Foreign key attribute:

If a key attribute of one table acts like non key attribute in other table than that attribute is known as Foreign key attribute.

dept: deptno,dname,loc ---> deptno is key attribute

emp: empno,ename,sal,comm,job,hiredate ,deptno ----> deptno is non key attribute

example:

deptno is a foreign key attribute

product--> pid,pname,price
customer--> cid,cname,phone,pid

Functional Dependency:

It is used to identify the relation between attributes of a table.

R(X,Y) Relation between X and Y attributes (X and Y are the attributes)
X->Y X determines Y --->(X attribute is used identify the Y attribute)
then,
X is known as Determinant
Y is known as Dependent.

Types of dependency are given below:

1. Total Functional Dependency/Fully Functional Dependency
2. Partial Functional Dependency
3. Transitive Functional Dependency

1. Total Function dependency:

In a relation between attributes of a table all the non key attributes of table determined by a single key attribute then it is known as Total Functional dependency.

R(L,M,N)
L is the key attribute
M,N are non key attribute
L->M and L->N , here M and N are dependent on L attribute

R(empno,ename,sal,job,hiredate,comm,deptno)

empno is a key attribute
and ename,sal,job hiredate,comm,deptno are non key attribute

in this relation empno attribute used to identify all the non key attribute of emp table uniquely and their exists a Total Functional Dependency or Fully Functional Dependency.

2. Partial Functional Dependency:

R(a,b,c,d)

(a,c)---> composite key attribute

(a,c)->b

(a,c)->d

a->b (a is the part of composite key identifying the non key attribute call b uniquely)

It is a type of functional dependency at which a part of composite key identifying the another non key attribute uniquely it is known as Partial Functional Dependency.

example:

R(father,mother,son,daughter)

(father,mother)---> composite key

mother-> son

3. Transitive Functional Dependency:

If one non key attribute determines another non key attribute uniquely which in turn determined by the key attribute it is known Transitive Functional Dependency.

example:

R(l,m,n)

l is a key attribute

m,n are non key attributes.

l->m, m->n

l->n

R(father,mother,son,daughter)

father-->key attribute

mother,son,daughter---> non key attribute

father->daughter daughter-> son

father->son

Normalization:

It is the process of dividing the larger table into smaller table in order to remove data redundancy and Anomalies.

---> the process of decomposing the larger table into smaller table in order to remove Redundancy and Anomalies.

Normalized table:

A table without Anomalies and Redundancy are known to be a normalized table.

Anomalies:

These are side effect that is caused due to DML operations.

Redundancy:

Repeated/duplicate records on table are known to a redundancy.

To take to divide the large table to a smaller table in order to make a table as a Normalized table we go for levels of Normalization.

Normal Form:

A table without redundancy and Anomalies are known as Normal Form.

when a table reaches 3NF by default table will be in the Normal Form.

Following are levels of Normalization.

1. First Normal Form(1NF)
2. Second Normal Form(2NF)
3. Third Normal Form(3NF)
4. (Boyce Codd Normal Form(BCNF)

1. First Normal Form(1NF)

if a table wants to be in 1NF it has to follow below rules.

1. The table should not have any Duplicate records
2. The table should not have any multivalued data

2. Second Normal Form(2NF)

If a table wants to be in the 2NF it has to follow Rules:

1. the table should be in the 1NF
2. the table should not have a Partial Function Dependency

cid	cname	course	c_fees
1	AJAY	SQL	1500
2	VINAY	JAVA	2000
3	JEMIMA	HTML	1200
4	RESHMA	SQL	1500
1	AJAY	HTML	1200
5	PADMA	JDBC	4000
6	SONALI	PYTHON	3000
2	VINAY	JDBC	4000

(cid,course)-->composite key

course->c_fees

cid	cname	course (f.k)
1	AJAY	SQL
2	VINAY	JAVA
3	JEMIMA	HTML
4	RESHMA	SQL
1	AJAY	HTML
5	PADMA	JDBC
6	SONALI	PYTHON
2	VINAY	JDBC

P.K course	c_fees
SQL	1500
JAVA	2000
HTML	1200
JDBC	4000
PYTHON	3000

3. THIRD NORMAL FORM:

If a table wants to be in a 3NF it should follow below rules.

1. the table should be in a 2NF
2. the table should not have any Transitive Functional Dependency

cid->city city->pincode

cid->pincode

cid	cname	city	pincode
1	Vidya	Pune	411041
2	Manish	Odisha	770032
3	Yashu	Hyderabad	500010
4	Abdullah	Pune	411041
5	Reshma	Chennai	305112
6	Sonali	Odisha	770032

cid	cname	city
1	Vidya	Pune
2	Manish	Odisha
3	Yashu	Hyderabad
4	Abdullah	Pune
5	Reshma	Chennai
6	Sonali	Odisha

city	pincode
Pune	411041
Odisha	770032
Hyderabad	500010
Chennai	305112

BCNF(Boyce Codd Normal Form):

It is just update a version of 3rd Normal Form

---> It is also known as 3.5NF